

Thary: Sistema de apoyo a la decisión basado en predicción de demanda para la gestión de inventarios

Danna Andrea Rodriguez Galarcio

Trabajo de grado presentado para optar al título de ingeniero de sistemas

Asesor:

Ivan Mauricio Cabezas



Universidad de San Buenaventura
Facultad de Ingeniería (Cali)
Ingeniería Sistemas
Santiago de Cali, Colombia
2026

Citar/How to cite	Nombre . [1]
Referencia/Reference	[1] Nombre . “Thary: Sistema de apoyo a la decisión basado en predicción de demanda para la gestión de inventarios”, Trabajo de grado profesional , Ingeniería Sistemas , Universidad de San Buenaventura Santiago de Cali, Colombia , 2026 .
Estilo/Style:	
IEEE (2020)	



Biblioteca Digital (Repositorio)
www.bibliotecadigital.usb.edu.co

Bibliotecas Universidad de San Buenaventura

Biblioteca Fray Alberto Montealegre O.F.M. - Bogotá.

Biblioteca Fray Arturo Calle Restrepo O.F.M. - Medellín, Bello, Armenia, Ibagué.

Departamento de Biblioteca - Cali.

Biblioteca Central Fray Antonio de Marchena – Cartagena.

Universidad de San Buenaventura Colombia - www.usb.edu.co

Bogotá - www.usbbog.edu.co

Medellín - www.usbmed.edu.co

Cali - www.usbcali.edu.co

Cartagena - www.usbctg.edu.co

Editorial Bonaventuriana - www.editorialbonaventuriana.usb.edu.co

Revistas científicas – www.revistas.usb.edu.co

Dedicatoria

Texto de dedicatoria centrado.

Agradecimientos

Texto de agradecimientos centrado.

TABLA DE CONTENIDO

Resumen	9
Abstract	10
Glosario	11
1 Capítulo 1: Introducción	13
1.1 Contexto	13
1.2 Planteamiento del Problema	13
1.3 Justificación del Problema	14
1.3.1 Impactos	14
1.4 Propuesta de Solución	15
1.5 Pregunta de Investigación	16
1.5.1 Preguntas Derivadas	16
1.6 Objetivos	17
1.6.1 Objetivo General	17
1.6.2 Objetivos Específicos	17
1.7 Restricciones	17
1.8 Recursos y Viabilidad Técnica	18
2 Capítulo 2: Marco de Referencia	19
2.0.1 Criterios de inclusión	19
2.0.2 Criterios de exclusión	19
2.1 Fundamentos Teóricos	19
2.1.1 Gestión de inventarios	19
Tipos de gestión de inventario	20
2.1.2 Predicción de demanda	20
2.1.3 Series de tiempo	21
2.1.4 Modelos de predicción de demanda	21
Redes Neuronales Artificiales (RNA)	21
Long Short-Term Memory (LSTM)	21
Gated Recurrent Unit (GRU)	22
Modelos estadísticos y de Machine Learning tradicionales	23
Regresión lineal	23
Modelos basados en árboles de decisión	23

	XGBoost	23
2.2	Antecedentes	24
2.2.1	Comparación de modelos de predicción de demanda	24
2.3	Estándares de Ingeniería Aplicables	25
3	Capítulo 3: Diseño y Desarrollo	26
3.1	Enfoque Metodológico	26
3.1.1	Gestión del Proyecto (Planeación)	26
3.1.2	Ciclo de Vida del Software (Ejecución)	27
3.2	Procesos Técnicos y Diseño (ISO/IEC/IEEE 12207)	27
3.2.1	Identificación de Stakeholders	28
3.2.2	Elicitación de Requisitos	29
3.2.3	Especificación de Requisitos de Software (SRS)	30
3.2.4	Diseño de la Arquitectura	33
	Escenarios Arquitecturales	34
	Análisis de Alternativas y Decisiones de Diseño (ADRs)	35
	Representación de la Arquitectura (Modelo C4)	36
3.2.5	Diseño Detallado	38
	Modelo de Datos	38
	Diseño UX/UI	41
3.2.6	Análisis de Amenazas y Diseño Seguro	42
3.3	Construcción, Integración y Despliegue	43
	Construcción	43
	Integración	44
	Integración continua (CI)	47
	Despliegue	48
3.4	Plan de Verificación y Validación	51
	Estrategia de verificación	51
	Estrategia de validación del modelo de predicción	51
	Criterios de aceptación	51
	Plan de validación con usuarios	51
4	Capítulo 4: Análisis de Resultados y Juicio de Ingeniería	52
4.1	Resultados de Verificación (Pruebas de Software)	52
4.2	Resultados de Validación (Evaluación del Usuario)	52
4.2.1	Validación del modelo de predicción	52
4.2.2	Consistencia mediante validación cruzada temporal	53

4.3 Evaluación de la Seguridad y Estándares	56
4.4 Análisis de Impactos de la Solución	56
4.5 Conclusiones	56
4.6 Recomendaciones de Operación y Mantenibilidad	56
4.7 Trabajos Futuros	56
4.8 Declaración de Uso Responsable de IA	56
Referencias Bibliográficas	57
ANEXOS	58
Anexo A Documento completo SRS (historias de usuario / backlog)	59
Anexo B Modelos de datos extensos, diccionarios de datos, mockups	68
Anexo C Manual de usuario y/o guía de instalación y despliegue	69
Anexo D Enlaces a repositorios de código fuente y tableros de gestión (Azure DevOps, Jira, GitHub)	70
Anexo E Protocolo y seguimiento de la revisión sistemática de literatura (si aplica)	71

LISTA DE TABLAS

I	Comparación de técnicas de predicción aplicadas a problemas de demanda y series de tiempo	25
II	Análisis de interés e influencia de los stakeholders	28
III	Trazabilidad entre casos de uso y requisitos funcionales	32
IV	Entidades principales del modelo de datos.	39
V	Métricas del holdout final por modelo.	52
VI	Resumen de la validación cruzada temporal (promedio $\pm$ desviación, 3 folds). . . .	55
VII	Especificación del caso de uso CU-01: Iniciar sesión	59
VIII	Especificación del caso de uso CU-02: Cargar historial de consumo	60
IX	Especificación del caso de uso CU-03: Procesar archivo	61
X	Especificación del caso de uso CU-04: Consultar demanda pronosticada	62
XI	Especificación del caso de uso CU-05: Consultar estado del inventario	63
XII	Especificación del caso de uso CU-06: Consultar recomendaciones de reposición y presupuesto	64
XIII	Especificación del caso de uso CU-07: Registrar pedido	65
XIV	Especificación del caso de uso CU-08: Gestionar usuarios	66
XV	Especificación del caso de uso CU-09: Consultar asistente virtual	67

LISTA DE FIGURAS

1	Esquema detallado de entradas, proceso y salidas del sistema.	14
2	Esquema detallado de la propuesta de solución.	16
3	Caso de uso	31
4	Diagrama de contexto del sistema (nivel 1 del modelo C4).	36
5	Diagrama de contenedores del sistema (nivel 2 del modelo C4).	37
6	Diagrama de componentes del sistema (nivel 3 del modelo C4).	37
7	Servicios desplegados en Railway: aplicación web (Thary) y base de datos MySQL.	48
8	Detalles de construcción y despliegue del servicio Thary en Railway.	49
9	Variables de entorno del servicio Thary en Railway, con los valores ocultos.	50
10	Pantalla de inicio de sesión de Thary, accedida desde su URL pública en Railway.	51
11	Comparación de WAPE entre los tres modelos candidatos (holdout final).	53

Resumen

resumen *Palabras clave* — **Predicción de demanda, gestión de inventarios, sistemas de apoyo a la decisión, series de tiempo**

El presente proyecto tiene como objetivo desarrollar un sistema de apoyo a la decisión orientado a la predicción de la demanda de productos en los inventarios.

Esta problemática surge a partir de la falta de información precisa sobre las cantidades necesarias de productos, lo cual dificulta la correcta planificación y puede llevar a que halla desabastecimiento, afectando negativamente la atención de los clientes.

Para abordar esta situación, en este trabajo se propone el uso de modelos de predicción basados en datos históricos de consumo. El sistema permitirá estimar las cantidades requeridas de productos cada mes, así como generar indicadores que apoyen la toma de decisiones para los encargados del manejo del inventario, tales como puntos de reposición y alertas.

Metodológicamente, se emplean diferentes técnicas y enfoques de la ingeniería de sistemas para obtener esta herramienta funcional, que contribuya a mejorar la gestión del inventario, optimizar la planificación de compras y reducir el riesgo de desabastecimiento.

Abstract

Abstract *Keywords* — **Demand forecasting, inventory management, decision support systems, time series** The objective of this project is to develop a decision support system designed to predict demand for products in inventory.

This problem arises from a lack of accurate information regarding the required quantities of products, which hinders proper planning and can lead to shortages, negatively affecting customer service.

To address this situation, this study proposes the use of forecasting models based on historical consumption data. The system will enable the estimation of the required quantities of products each month, as well as the generation of indicators to support decision-making for inventory managers, such as reorder points and alerts.

Methodologically, various techniques and approaches from systems engineering are employed to develop this functional tool, which will help improve inventory management, optimize purchasing planning, and reduce the risk of stockouts.

Glosario

Demanda: Cantidad de productos requeridos durante un período determinado, calculada a partir del comportamiento histórico de consumo o ventas.

Predicción de demanda: Proceso mediante el cual se estiman las cantidades futuras de productos utilizando modelos estadísticos o de aprendizaje automático basados en datos históricos.

Gestión de inventarios: Conjunto de actividades orientadas al control, almacenamiento, abastecimiento y seguimiento de los productos disponibles en una organización.

Inventario Conjunto de productos, materiales o recursos almacenados y disponibles para su uso, venta o distribución.

Desabastecimiento: Situación en la que un producto no se encuentra disponible en el inventario cuando es requerido.

Sistema de apoyo a la decisión: Herramienta informática diseñada para asistir a los usuarios en la toma de decisiones mediante el análisis de datos y la generación de información relevante.

Modelo de predicción: Algoritmo o método matemático utilizado para estimar valores futuros a partir de datos históricos y patrones identificados en la información.

Datos históricos: Registros de consumo, ventas, compras o movimientos de inventario ocurridos en períodos anteriores y utilizados para el análisis y la predicción.

Punto de reposición: Nivel de inventario a partir del cual se debe realizar un nuevo pedido para evitar el desabastecimiento.

Tiempo de entrega (Lead Time): Período que transcurre entre la realización de un pedido y la recepción efectiva de los productos solicitados.

Serie de tiempo: Conjunto de observaciones registradas en orden cronológico que permiten analizar patrones, tendencias y comportamientos a lo largo del tiempo.

Aprendizaje automático (Machine Learning): Rama de la inteligencia artificial que permite a los sistemas aprender patrones a partir de datos y realizar predicciones sin ser programados explícitamente para cada situación.

Dominio de inventario: Contexto o sector específico en el que se gestionan inventarios, como

retail, manufactura, logística, salud, restaurantes o comercio electrónico.

1 Capítulo 1: Introducción

1.1 Contexto

La gestión eficiente de inventarios supone un desafío constante en diversos sectores. Problemas como sufrir de debastabecimiento pueden surgir por distintos factores como retrados en la entrega por parte de los proveedores, variaciones inesperadas de la demanda, o deficiencias en los procesos de planificación. Estas situaciones, en muchos casos, se encuentran fuera del control de los administradores, lo que incrementa la complejidad para ellos a la hora de tomar decisiones relacionadas con el abastecimiento.

Actualmente, normalmente en muchas empresas se cuenta con un software para gestión de inventarios tipo ERP, con este software se administra la disponibilidad de los productos mediante el uso de diferentes tipos de movimientos de inventario:

Uno de los tipos de movimientos consiste en las entradas de inventario, las cuales responden a la adquisición de productos a través de proveedores y son registradas mediante facturas que incluyen información como fecha, proveedor y costo.

Por otro lado, están las salidas por consumo, las cuales representan la venta, distribución o consumo de los productos, registrando información relacionada con las cantidades entregadas y el destino de estas. Adicionalmente, existen salidas de inventario asociadas a sucesos como devoluciones, productos dañados, vencimientos, pérdidas o ajustes de inventario, se registra la cantidad de productos que deben de ser retirados del inventario junto con su respectivo motivo. Posteriormente, se realizan ajustes de inventario, los cuales pueden ser tanto positivos como negativos, con el fin de corregir diferencias entre las cantidades registradas en el sistema y las existencias reales, garantizando así la consistencia de la información.

En general, un buen análisis de estos movimientos resulta fundamental para comprender el comportamiento del consumo de productos y así mejorar los procesos de planificación del abastecimiento.

1.2 Planteamiento del Problema

En diferentes entornos, existe una problemática en la que la gestión del inventario de productos se ve afectado debido a la falta de información precisa sobre la demanda mensual de los mismos, lo que

dificulta la planificación adecuada del abastecimiento. En este contexto, surge la necesidad de contar con herramientas que permitan analizar los datos históricos de inventario y generar estimaciones de demanda que apoyen la toma de decisiones relacionadas con el abastecimiento.

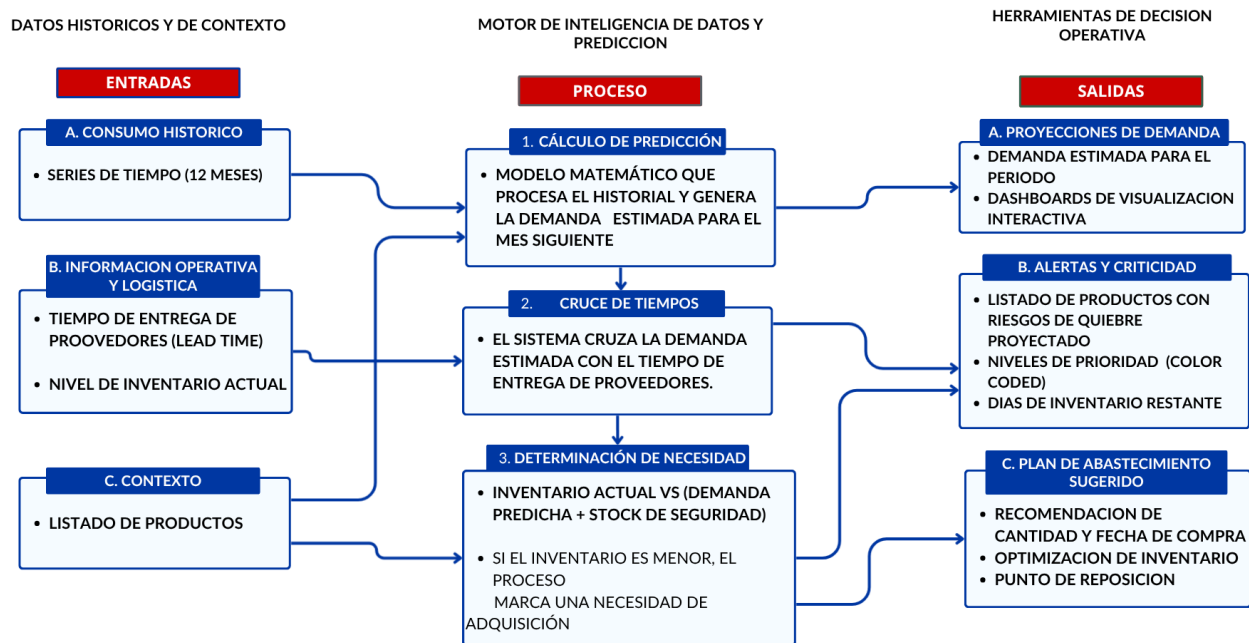


Fig. 1. Esquema detallado de entradas, proceso y salidas del sistema.

1.3 Justificación del Problema

Este proyecto se justifica teniendo en cuenta que esta es una problemática que no se ha visto abordada a profundidad y continua siendo una dificultad para organizaciones de diferentes sectores. Además este proyecto permite aplicar conocimientos de la ingeniería de sistemas como análisis de datos, modelos de predicción y aprendizaje automático para la construcción de una herramienta que contribuya a mejorar los procesos de planificación y administración, generando beneficios tanto para las organizaciones y los usuarios finales.

1.3.1 Impactos

Mejorar la disponibilidad de los productos mediante una mejor planificación de los inventarios, apoyar la toma de decisiones del personal encargado del abastecimiento, y fortalecer el uso de herramientas tecnológicas en el sector administrativo.

1.4 Propuesta de Solución

Con este proyecto se propone el desarrollo de un software que ayude a predecir la demanda de productos en diferentes dominios y contextos de gestión de inventarios. Este sistema se realizará tras analizar datos como historial de consumo, esto para poder estimar exactamente las cantidades necesarias de cada producto en periodos mensuales.

La solución implementará modelos de predicción de demanda que procesarán la información histórica para identificar patrones y en base a ellos, generar estimaciones futuras. A partir de estas predicciones, el sistema podrá proporcionar información relevante para la toma de decisiones, tales como cantidades recomendadas de compra, puntos de reposición y posibles alertas de desabastecimiento.

Se utilizarán datos reales del año 2021 del sector salud como caso de estudio, que serán complementados con datasets con datos públicos de diferentes dominios esto para poder evaluar el comportamiento del sistema en diferentes contextos.

Adicionalmente, el sistema tendrá en cuenta variables como tiempo de entrega de proveedores (lead time) y cantidad de productos disponibles, con el propósito de lograr que el sistema sea lo más preciso posible.

Esta información será presentada en una interfaz que permitirá visualizar los resultados de manera comprensible, esto para facilitar su uso por parte del personal encargado de la gestión de inventarios.

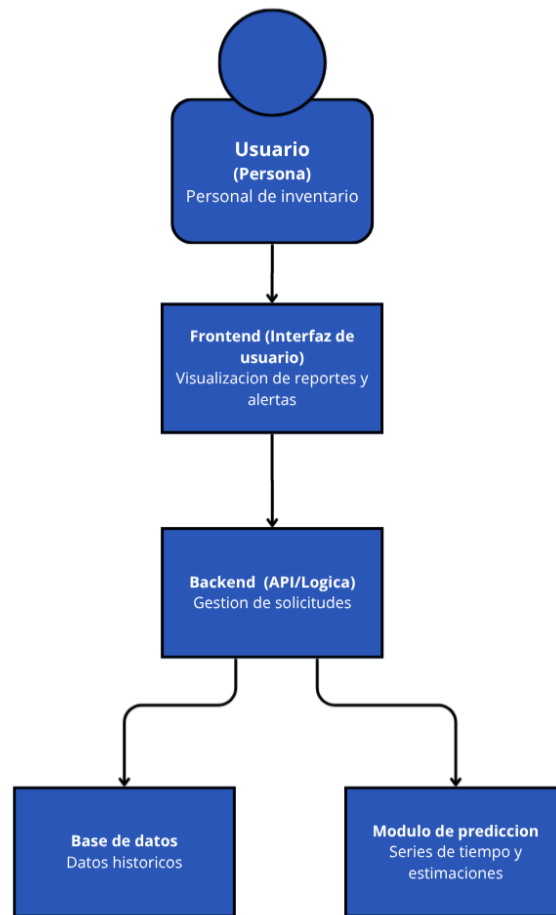


Fig. 2. Esquema detallado de la propuesta de solución.

1.5 Pregunta de Investigación

¿Cómo se puede diseñar e implementar un sistema de apoyo a la decisión que, mediante modelos de predicción de demanda, permita estimar las cantidades necesarias de productos en un sistema de gestión de inventarios?

1.5.1 Preguntas Derivadas

¿Qué variables y datos históricos son necesarios para estimar la demanda mensual de productos?

¿Qué modelos de predicción de demanda son más adecuados para estimar el consumo o venta de productos en diferentes dominios?

¿Cómo debe estructurarse el proceso de análisis y procesamiento de datos para generar

predicciones confiables?

¿Qué tipo de salidas debe generar el sistema (por ejemplo, cantidades estimadas, punto de reposición, alertas) para apoyar la toma de decisiones?

1.6 Objetivos

1.6.1 Objetivo General

Desarrollar un sistema que le permita a los usuarios poder predecir cual va a ser la demanda mensual de productos mediante el analisis de datos historicos, para garantizar su correcta planificacion y abastecimiento

1.6.2 Objetivos Específicos

Identificar las fuentes de datos relevantes, incluyendo el historial de consumo y las variables del inventario necesarias para la predicción de la demanda de productos.

Seleccionar modelos de predicción de demanda adecuados para estimar el consumo o venta mensual de productos en diferentes contextos de los inventarios.

Incorporar variables como el tiempo de entrega de proveedores y el punto de reposición dentro del sistema para complementar las estimaciones de demanda.

Diseñar un módulo de visualización que presente las predicciones de manera clara y comprensible para apoyar la toma de decisiones del usuario.

Evaluar el desempeño del sistema desarrollado mediante la realización de pruebas con datos históricos, con el fin de analizar la precisión de las predicciones generadas y verificar su utilidad en el apoyo a la toma de decisiones en la gestión del inventario.

1.7 Restricciones

El proyecto tiene como principal alcance el desarrollo de un sistema de apoyo a la decisión orientado a la predicción de la demanda de medicamentos críticos en una farmacia hospitalaria, esto utilizando los datos históricos de consumo como principal fuente de información. El sistema permitirá estimar la cantidad mensual requerida de medicamentos mediante la implementación de

modelos de predicción de demanda, integrando funcionalidades como visualización de resultados, generación de reportes y apoyo en la toma de decisiones relacionadas con la planificación del abastecimiento. Asimismo, el sistema proporcionara al usuario cálculos como el punto de reposición, con el fin de contribuir a una mejor organización y control de los recursos disponibles en los entornos hospitalario.

El desarrollo del proyecto presenta algunas restricciones que deben ser consideradas. En primer lugar, el sistema se basará en un conjunto de datos históricos reales de la Clínica Nuestra señora de los remedios limitado al año 2021, lo que puede afectar la capacidad del modelo para capturar comportamientos actuales o cambios en la demanda a lo largo del tiempo. Además, no se contará con integración en tiempo real con sistemas hospitalarios, por lo que las predicciones se realizarán sobre datos estáticos.

1.8 Recursos y Viabilidad Técnica

2 Capítulo 2: Marco de Referencia

Para la construcción del estado del arte, se realizó una revisión de literatura en bases de datos académicas reconocidas, con el fin de identificar estudios relevantes relacionados con la predicción de demanda y la gestión de inventarios en el sector salud.

2.0.1 Criterios de inclusión

Se consideraron los siguientes criterios para la selección de los estudios:

- Trabajos disponibles en español o inglés.
- Publicaciones académicas en bases de datos como IEEE Xplore, Scopus, ScienceDirect, Google Scholar y repositorios institucionales.
- Estudios enfocados en gestión de inventarios.
- Investigaciones que utilicen modelos de predicción de demanda.
- Estudios que describan el diseño de sistemas informáticos de apoyo a la decisión.

2.0.2 Criterios de exclusión

Se excluyeron los estudios que cumplieran con las siguientes características:

- Estudios con más de 10 años de antigüedad.
- Documentos incompletos o resúmenes sin acceso al texto completo.

2.1 Fundamentos Teóricos

2.1.1 Gestion de inventarios

La gestion de inventarios consiste en el proceso que se realiza sobre la mercancía para conocer la cantidad disponible, esto permitiendo analizar los niveles de existencia y la aproximación del consumo real.[1]

Los pasos basicos para realizar una correcta gestion de inventario consiste en: Compra del inventario, en el que los productos se entregan al almacen o punto de venta, Almacenamiento del inventario, en el que este se guarda hasta que se necesite y Aprovechamiento del inventario, en donde se controla la cantidad de productos que hay a la venta para finalmente enviarlo o venderlo a los clientes.[2]

Tipos de gestion de inventario:

Gestión periódica de inventarios Este método implica realizar un recuento físico del inventario a intervalos específicos. Se toma el inventario al inicio del periodo, se suman las nuevas compras y se deduce el inventario final para calcular el costo de las mercancías vendidas (COGS).[2]

Gestión de inventario de códigos de barras Utiliza un sistema que asigna un número único a cada producto. Permite asociar datos como proveedor, dimensiones del producto, peso y cantidad en stock.[2]

Gestión de inventario RFID Transmite de forma inalámbrica la identidad de un producto mediante un número de serie único. Mejora la eficiencia y visibilidad del inventario, facilitando el seguimiento de artículos.[2]

2.1.2 Prediccion de demanda

La prediccion de la demanda es un proceso que permite mejorar la gestion de los niveles de inventario mediante la utilizacion de datos historicos. Una correcta prediccion de la demanda le permite a las organizaciones el poder tomar decisiones mas inteligentes a la hora de manejar sus provisiones, evitando la falta o exceso de productos, de esta manera, aumentando tanto la satisfaccion del cliente como la creacion de estraregias de negocio mas efectivas a corto y largo plazo [3].

En los ultimos años, los mercados han evolucionado y se han vuelto cada vez mas exigentes en cuanto la calidad y precision de los servicios, es por eso, que muchas organizaciones se ha visto inclinadas a incluir el uso de nuevas herramientas y tecnologias como Inteligencia Artificial. Machine Learning o analisis predictivo, para automatizar y perfeccionar esta tarea, ya que los metodos actuales estaticos que son la norma en muchas empresas y organizaciones han empezado a quedar obsoletas para satisfacer las necesidades del mercado actual.[3], [4]

La principal debilidad de los modelos de inventario actuales es que al ser poco robustos, no

toman en contemplación diferentes factores externos de manera inteligente [4].

2.1.3 Series de tiempo

Las series temporales son usadas para analizar datos históricos recogidos en intervalos regulares, como ventas diarias o mensuales, para identificar patrones, tendencias estacionales y ciclos. Estos patrones se identifican mediante técnicas como gráficos ACF y PACF, también estos patrones sirven para ajustar modelos estadísticos o de aprendizaje automático (como ARIMA, SARIMAX o Prophet) que permiten proyectar valores futuros [5]

Se utilizan estos modelos para hacer predicciones, los datos se agrupan en intervalos (mensualmente) para reducir variabilidad y facilitar el análisis. Tras el análisis de datos, se ajustan los modelos y se evalúan mediante métricas como el RMSE, para que las predicciones sirvan en la toma de decisiones de gestión y planificación de recursos

2.1.4 Modelos de predicción de demanda

Redes Neuronales Artificiales (RNA):

Las redes neuronales artificiales (RNA) son técnicas útiles para la previsión de la demanda, gracias a su capacidad para manejar datos no lineales y captar relaciones funcionales sutiles entre datos empíricos, incluso cuando las relaciones subyacentes son desconocidas o difíciles de describir.

Long Short-Term Memory (LSTM)

Las redes neuronales de memoria a largo y corto plazo o LSTM es un tipo de red neuronal que se ha demostrado eficaz en la predicción de series temporales, esta técnica está diseñada para procesar datos secuenciales y temporales que fue creada para superar las limitaciones de las redes neuronales tradicionales.[6].

Su funcionamiento se basa en una estructura de memoria que permite a la red aprender y recordar información a lo largo del tiempo, lo que es especialmente útil para capturar dependencias a largo plazo en los datos. LSTM cuenta con las tres siguientes puertas:

Puerta de olvido (forget gate): Decide qué información se descarta. Esta se calcula mediante la función sigmoide, que toma como entrada el vector de entrada actual y el estado oculto anterior, de esta manera, generando un valor entre 0 y 1 para cada componente de la celda de la memoria.

$$f_t = \delta(W_f[h_{t-1}, X_t] + b_f) \quad (1)$$

Puerta de entrada (input gate): Decide qué información se almacena en la memoria. También usa una función tanh para normalizar la información que pasa a través de la red neuronal.

$$i_t = \delta(W_i[h_{t-1}, X_t] + b_i), \quad L_t = \tanh(W_c[h_{t-1}, X_t] + b_c) \quad (2)$$

Puerta de salida (output gate): Decide qué información se utiliza para generar la salida oculta actual.

$$o_t = \delta(W_o[h_{t-1}, X_t] + b_o) \quad (3)$$

La actualización del estado celular se realiza mediante:

$$C_t = f_t * C_{t-1} + i_t * L_t \quad (4)$$

Finalmente, la salida oculta se calcula aplicando la función tanh al estado celular y multiplicándolo por la puerta de salida como:

$$h_t = o_t * \tanh(C_t) \quad (5)$$

Gated Recurrent Unit (GRU)

GRU es un modelo de predicción bastante parecido a LSTM, pero con una arquitectura mucho más simple, lo que la hace muchísimo más compacta y potencialmente menos costosa computacionalmente que una LSTM equivalente. Estas han demostrado tener éxito en diferentes aplicaciones relacionadas con datos secuenciales o temporales, principalmente si son secuencias largas. GRU reduce el número de tres redes compuestas distintas que usa LSTM en solo 2, esto mediante la combinación de la compuerta de entrada y la compuerta de olvido mediante una suma directa. Las compuertas de GRU se denominan: Compuerta de actualización y Compuerta de reinicio. [7]

El modelo GRU se presenta como:

$$h_t = g(Uh(r_t \odot h_{t-1}) + Wx_t + b_h) \quad (6)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (7)$$

donde z_t es la compuerta de actualización, r_t es la compuerta de reinicio y $\tilde{h}_t$ es la propuesta de nueva información. Las dos compuertas se presentan como:

$$z_t = \sigma(U_z h_{t-1} + W_z x_t + b_z) \quad (8)$$

$$r_t = \sigma(U_r h_{t-1} + W_r x_t + b_r) \quad (9)$$

Modelos estadísticos y de Machine Learning tradicionales:

Regresión lineal

Los modelos de regresion son utilizados para resolver problemas de prediccion, el objetivo de este modelo puede ser explicativo, donde se busca, por ejemplo, evaluar como ciertas variables independientes afectan a una variable dependiente, o predictivo, en el que se busca estimar el valor de la variable dependiente en funcion de las independientes [8].

Un modelo de regresion se expresa como:

$$y = (x_1, x_2, \dots, x_n)\beta + \epsilon \quad (10)$$

Existen distintas opciones para poder estimar un modelo de regresion, siendo las mas comunes: Regresion lineal y regresion logistica. La eleccion depende del tipo de variable dependiente que se tenga, si es una variable continua se utiliza regresion lineal, mientras que si se trata de una variable dicotomica se utiliza una variable dicotomica.

Modelos basados en árboles de decisión:

XGBoost

XGBoost se ha utilizado ampliamente en numerosos campos para lograr resultados de vanguardia (state-of-the-art) en tareas relacionadas con datos y predicción. Su funcionamiento consiste en un sistema de aprendizaje automatizado basado en el metodo de Tree Boosting que incluye un

algoritmo de aprendizaje de arboles. EL Tree Boosting es un algoritmo de aprendizaje por conjuntos o ensemble learning, capaz de transformar varios clasificadores debiles en uno fuerte con el objetivo de obtener un mejor rendimiento en le clasificacion, es decir, se combinan multiples arboles para obtener un modelo mas potente. Un modelo tree boosting se escribe de la siguiente manera[9]:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), f_k \in F \quad (11)$$

Donde:

$$F = \{f(x) = w_{q(x)}\}(q : R^m \rightarrow T, w \in R^T) \quad (12)$$

Representa el espacio de los arboles de regresion o clasifiacion

2.2 Antecedentes

2.2.1 Comparación de modelos de predicción de demanda

Debido a que existen diferentes modelos y tecnicas para la prediccion de la demanda, se realizo una revision de literatura de diferentes estudios que emplean los modelos mas utilizados para la prediccion de la demanda, esto con el fin de realizar el siguiente analisis comparativo para determinar cual es el mas adecuado para la prediccion de demanda en el presente proyecto, teniendo en cuenta ventajas, limitaciones y características como: tipo de datos utilizados, dominio de aplicación, tecnica de aplicacion, metricas de evaluacion y resultados obtenidos, dataset utilizado, entre otros.

TABLA I

Comparación de técnicas de predicción aplicadas a problemas de demanda y series de tiempo

Aspecto	Redes neuronales (ANN)	LSTM	Regresión lineal	GRU	XGBoost
Referencia	<i>Demand Forecasting Using Neural Network for Supply Chain Management</i>	<i>LSTM Model Based Demand Forecasting in Supply Chain Using Efficient Optimizer</i>	<i>Application of ABC Analysis and Linear Regression Model in Inventory Management</i>	<i>StockNet-GRU Based Stock Index Prediction</i>	<i>XGBoost-Based Demand Forecasting in Supply Chain Management Using Machine Learning Algorithm</i>
Datos utilizados	Datos históricos de ventas organizados como series de tiempo. Se emplean ventas mensuales de un filtro de combustible entre 2011 y 2013 para predecir la demanda de 2014.	Datos históricos de ventas organizados como series de tiempo. Incluyen fecha, unidades vendidas, ganancias y ubicación, seleccionando las variables relevantes para el entrenamiento.	Datos históricos de inventario y ventas. Incluyen información de productos, inventario, órdenes de compra y venta, cantidad y precio de venta, fecha, marcas, clasificación e impuestos.	Series de tiempo financieras correspondientes a precios históricos de apertura del índice bursátil CNX Nifty y de las 10 acciones con mayor correlación. Los datos se normalizan y organizan mediante ventanas deslizantes de 20 días.	Datos logísticos y de cadena de suministro relacionados con la demanda, incluyendo variables asociadas con proveedores y logística. Los datos se normalizan mediante Z-score y posteriormente se seleccionan las variables relevantes.
Dominio	Gestión de la cadena de suministro, control de inventarios, manufactura y distribución. También presenta aplicaciones en turismo, consumo eléctrico, supermercados y ventas.	Gestión de la cadena de suministro y predicción de demanda.	Gestión de inventarios, cadena de suministro y análisis de ventas.	Predicción de índices bursátiles, mercados financieros y análisis de series de tiempo.	Gestión de la cadena de suministro, predicción de demanda, logística y optimización de inventarios. El enfoque puede aplicarse a diferentes sectores que requieran pronósticos de demanda.
Técnica	Red neuronal <i>feed-forward</i> entrenada mediante <i>Backpropagation</i> . Se comparan Gradient Descent, Conjugate Gradient y Levenberg-Marquardt, siendo este último el de mejor desempeño.	Red neuronal recurrente LSTM con tres capas, regularización mediante Dropout y optimizadores Adam y SGD. Se experimenta con épocas, tasa de aprendizaje, tamaño de lote y longitud de secuencia.	Modelo de regresión lineal para identificar tendencias en cantidad y precio de venta. Utiliza Gradient Descent para minimizar el error cuadrático, después de realizar limpieza de datos y selección de variables.	Red neuronal recurrente GRU integrada en la arquitectura StockNet. Utiliza ventanas deslizantes de 20 días, optimizador Adam, activación ReLU y datos normalizados entre 0 y 1.	El proceso incluye normalización Z-score, selección de características mediante un Algoritmo Genético, predicción mediante XGBoost y optimización de hiperparámetros mediante Particle Swarm Optimization (PSO).
Métricas	Error absoluto de pronóstico (<i>Forecasting Error</i>) y porcentaje de error, comparando los valores pronosticados con los valores reales.	RMSE para medir la diferencia entre valores reales y predichos. También se compara el tiempo promedio por época para evaluar la eficiencia computacional.	Coefficiente de determinación (R^2). El estudio reporta un valor de 70,57 % para la predicción de la cantidad vendida y cercano al 80 % para el precio de venta.	RMSE, MAE y MAPE. También se considera el tiempo de ejecución y se utiliza la prueba estadística de Wilcoxon para validar diferencias entre modelos.	MAE, RMSE y coeficiente de determinación. Se compara el desempeño de XGBoost con SARIMA, Decision Tree (DT) y Random Forest (RF).
Dataset	Datos reales de consumo mensual de un filtro de combustible perteneciente a una empresa manufacturera. Contiene registros mensuales entre 2011 y 2013 para entrenamiento y validación.	<i>Sales Dataset</i> con información histórica de ventas utilizada para entrenar y evaluar el modelo.	Dataset de inventario de una tienda de comestibles (<i>grocery store</i>) con información de productos, inventario y ventas. Después del preprocesamiento se utilizan aproximadamente 60 observaciones correspondientes a dos meses.	Índice bursátil CNX Nifty y las 10 acciones más correlacionadas del National Stock Exchange (NSE) de India, obtenidos desde Yahoo Finance. Contiene 5.904 días de negociación entre abril de 1996 y enero de 2020.	Dataset de demanda de cadena de suministro con información relacionada con logística y proveedores. Se utiliza para entrenar y evaluar el modelo de predicción.
Comentario / reseña	Las ANN permiten modelar relaciones no lineales y presentan resultados favorables frente a métodos tradicionales de series de tiempo. Son útiles para predicciones de corto plazo, aunque requieren suficiente información y ajuste de parámetros.	LSTM constituye una alternativa eficaz para la predicción de demanda debido a su capacidad para capturar dependencias temporales de largo plazo. Puede contribuir a la optimización de inventarios y a la toma de decisiones en la cadena de suministro.	La regresión lineal permite identificar y proyectar tendencias generales de ventas e inventario y presenta una menor complejidad. Sin embargo, tiene limitaciones para representar relaciones complejas y eventos extraordinarios.	GRU permite capturar dependencias temporales y presenta resultados favorables en la predicción de series de tiempo. Sin embargo, el estudio analizado se desarrolla en el dominio financiero, por lo que su aplicación a inventarios requiere evaluación adicional.	XGBoost presenta capacidad para modelar relaciones complejas y puede utilizarse para apoyar la planificación de inventarios, reducir costos y optimizar operaciones en la cadena de suministro.

Fuente: Elaboración propia a partir de los estudios revisados.

2.3 Estándares de Ingeniería Aplicables

3 Capítulo 3: Diseño y Desarrollo

3.1 Enfoque Metodológico

El enfoque metodológico de este proyecto se basa en dos componentes principales. El primero se enfoca en determinar de manera empírica cual o cuales modelos de predicción de demanda resultan mas adecuados para el caso de estudio. El segundo componente se centra en la estructuración de dicho modelo dentro de una herramienta funcional, siguiendo procesos técnicos alineados con la norma ISO/IEC/IEEE 12207.

Para la evaluación de los modelos de predicción, el enfoque adoptado es empírico y por producto, ya que en lugar de seleccionar un único modelo ganador para la totalidad del catálogo de productos, se entrenan y comparan distintos modelos candidatos, y para cada producto, se selecciona de forma individual aquel que presente el menor error de pronóstico durante el periodo de prueba. Esta decisión responde a que no existe un modelo que universalmente pueda adaptarse a todos los patrones de demanda como: productos de consumo estable, esporádico o altamente variable, cada uno de los patrones puede favorecerse de distintas técnicas.

Debido a que los datos que fueron empleados corresponden a series de tiempo (registros de consumo mensual), se respeta el orden cronológico de la información. La separación entre los datos de entrenamiento y prueba se realiza por fecha, de manera que, se entrenan con los periodos mas antiguos y se evalúa sobre los mas recientes. Adicionalmente, se incorpora un procedimiento de validación cruzada que repite el proceso de entrenamiento y evaluación sobre distintas ventanas de tiempo sucesivas, con el fin de confirmar que el desempeño del sistema no dependa de las particularidades de un solo periodo de prueba.

Finalmente, el enfoque también incluye la evaluación mediante el uso de un caso de estudio real, ya que se usaran los datos históricos de consumo del año 2021 de la clínica Nuestra Señora de los Remedios, que constituye la validación del sistema en el dominio de la salud.

3.1.1 Gestión del Proyecto (Planeación)

La planeación del proyecto se estructura en cuatro fases secuenciales, cada una con un conjunto de entregables y actividades que se detallan a continuación:

- Fase 1 (Fase de fundamentación): En esta fase se plantean los objetivos del proyecto y realiza la revisión de literatura y construcción del marco de referencia, se definen los criterios de

inclusion y exclusion de los estudios consultados y se compara el desempeño reportado por los distintos modelos de prediccion de demanda.

- Fase 2 (Fase de analisis y diseño): Se definen los requisitos del sistema, la arquitectura y el modelo de datos
- Fase 3 (Fase de construccion): Se implementa el sistema de predicciom, con el modulo de calculo de puntos de reposicion y la interfaz de usuario, en esta fase se desarrollan el sistema identificando limitaciones en los datos o en los modelos, asi como limitaciones propias de la arquitectura inicial del sistema, las cuales se fueron corrigiendo a medida que avanzaba el desarrollo y surgian nuevas necesidades y propuestas para satisfacerlas.
- Fase 4 (Fase de evaluacion): En la fase final se le aplica al sistema un plan de verificacion y validacion, que incluye la evaluacion de los modelos de prediccion de demanda y la validacion del sistema en el dominio de la salud, mediante el uso de un caso de estudio real.

3.1.2 Ciclo de Vida del Software (Ejecución)

La ejecucion del proyecto se realizo siguiendo un ciclo de vida iterativo-incremental, en el cual el sistema se construyo en sucesicas versiones del mismo en el que se en cada una de estas se iba ampliando o corrigiendo detalles de la anterior a partir de la evaluacion de los resultados obtenidos. La eleccion de esta estrategia se basa en la naturaleza del proyecto, en el que al estar trabajando en el desarrollo de un sistema de prediccion, es esperable que decisiones como, la forma de representar los meses como variable de entrada, o el conjunto de variables historicas utilizadas por el modelo, no podrian definirse por adelantado, sino que se irian determinando y ajustando a medida que se experimentara y observara el comportamiento del sistema durante sus fases de desarrollo.

En cuanto al diseño de la arquitectura del sistema, el diseño de la interfaz de usuario, y los mecanismos de control de acceso al sistema, se siguieron los procesos tecnicos que indica la norma ISO/IEC/IEEE 12207, que establece un conjunto de procesos para a tener en cuenta como la identificacion de stakeholders, especificacion de requisitos, diseño arquitectonico, los cuales fueron definifos para el sistema.

3.2 Procesos Técnicos y Diseño (ISO/IEC/IEEE 12207)

Los procesos tecnicos que guiaron el diseño y desarrollo del sistema Thary se estructuraron siguiendo la ISO/IEC/IEEE 12207, una norma que establece un conjunto de procesos estandar para

el ciclo de vida del software. Dichos procesos fueron aplicados al desarrollo del sistema de la siguiente manera:

3.2.1 Identificación de Stakeholders

A partir del planteamiento del sistema y tras un análisis de sus características y funcionalidades, se identificaron los siguientes stakeholders que estaría involucrados en el sistema, ya sea directa o indirectamente:

- Personal encargado de la gestión del inventario y el abastecimiento: Es el usuario final que interactúa con el sistema, es responsable de interpretar los resultados que Thary y tomar decisiones de reposición en base a ellas.
- Equipo de desarrollo: Desarrolladores y diseñadores responsables de la implementación del sistema Thary, asegurando que cumpla con los requisitos y estándares establecidos.
- Administradores del sistema: Es el perfil con permisos y privilegios, es responsable de cargar los datos históricos de consumo, es decir, el archivo de entrada, ejecutar el proceso de predicción, además, puede crear nuevos usuarios para los empleados con menor rango.
- Institución usuaria del sistema: Es la entidad interesada en utilizar Thary para poder reducir el riesgo de desabastecimiento de productos. El sistema se planea para poder adaptarse en distintos dominios, por lo que cualquier tipo de empresa puede usarlo.
- Proveedores de productos: Aunque los proveedores no interactúan con Thary, indirectamente son una variable importante para el sistema, ya que factores externos como el tiempo de entrega o lead time, marcan una gran diferencia en los resultados del sistema.

TABLA II
Análisis de interés e influencia de los stakeholders

Stakeholder	Interés	Influencia
Personal de inventario y abastecimiento	Alto	Alta
Administrador del sistema	Alto	Alta
Institución usuaria	Alto	Alta
Proveedores de productos	Medio	Media
Equipo de desarrollo	Alto	Alta

3.2.2 Elicitación de Requisitos

La elicitation de requisitos consiste en poder identificar las necesidades del sistema a partir de los usuarios, en el caso de Thary, el proceso se llevo a cabo mediante dos fuentes principales de informacion, en primera instancia, la revision de literatura permitio analizar que variables suelen ser relevantes para este tipo de proyectos, y que salidas le aportan mayor valor al usuario. En segundo lugar, se realizo un analisis a un caso de estudio real, los datos usados durante el desarrollo del sistema eran datos autenticos de la Clinica Nuestra señora de los Remedios de la ciudad de Cali, lo cual permitio observar como se comportaria el sistema en un escenario real como la existencia de variaciones en la demanda o datos historicos no tan limpios como historiales incompletos o datos con errores, y que ajustes serian necesarios para mejorar su rendimiento.

Es en base a tdo esto, que se definieron los siguientes requisitos funcionales:

- RF-01: El sistema debe permitir que un usuario registrado como admin pueda realizar la carga de un archivo CSV con el datos como el historial de consumo mensual de los productos, validando que contenga los datos minimos requeridos: fecha, id del producto y cantidad demandada.
- RF-02: El sistema debe generar una estimacion de la demanda mensual, esperada para cada producto a partir de la informacion compartida en el historial cargado
- RF-03: El sistema debe de calcular para cada producto datos como el punto de reposicion y la cantidad recomendada de compra, considerando factores externos como el tiempo de entrega del proveedor o lead time, presupuestos y el inventario actual disponible
- RF-04: El sistema debe de contar con alertas visuales claras para los productos cuyo inventario actual se encuentre en o por debajo de su punto de reposicion.
- RF-05: El sistema debe presentar los resultados, demanda pronosticada, estado del inventario, etc, de una manera clara y sencilla de entender en una interfaz intuitiva para el usuario
- RF-06: El sistema debe permitirle al personal del inventario registrar un pedido, guardando informacion importante de este como proveedor elegido, fecha del pedido y el usuario responsable de la realizacion del pedido.
- RF-07: El sistema debe restringir el acceso a la carga de los archivos y la ejecucion del proceso unicamente al perfil administrador.

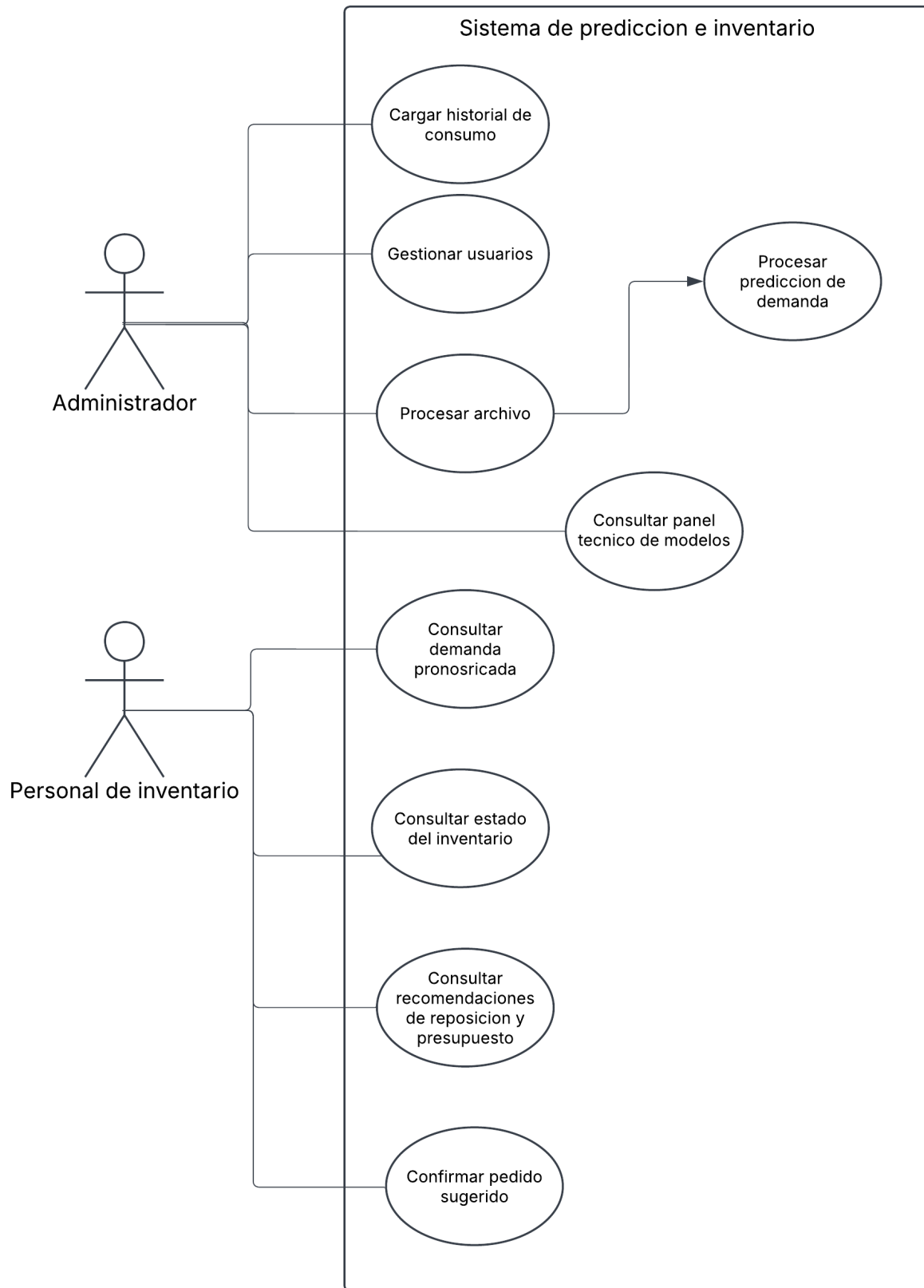
- RF-08: El sistema debe permitirle al perfil de administrador el poder crear nuevos usuarios con menos privilegios para sus empleados.
- RF-09: El sistema debe permitir el registro e inicio de sesión de los usuarios.
- RF-10: El sistema debe permitir a los usuarios consultar sus dudas sobre el funcionamiento del sistema mediante un asistente conversacional (chatbot), implementado utilizando la API de Ollama.

En cuanto a los requisitos no funcionales de Thary, se identificaron los siguientes:

- RNF-01: Si el sistema es utilizado por más de una organización los datos de una no deben de ser accesibles para otra
- RNF-02: El sistema debe de poder procesar archivos de distintos dominios de inventario (salud, alimentos, ferretería, etc) siempre y cuando contengan los datos mínimos requeridos.
- RNF-03: El sistema debe poder generar una estimación incluso si el historial de producto presenta datos faltantes.

3.2.3 Especificación de Requisitos de Software (SRS)

A partir de los requisitos especificados, se elaboró la especificación de requisitos funcionales (SRP), la cual fue estructurada mediante casos de uso y tomando como referencia los lineamientos de la norma IEEE 29148 para la elaboración de los mismos. El sistema contempla dos actores principales, el administrador y el personal de inventario. El administrador es el tipo de usuario que más privilegios tiene, es responsable de cargar el historial de consumo e iniciar el procesamiento y gestionar las cuentas de los usuarios. Mientras que el personal de inventario solo usa el sistema para consultar las predicciones y, entre otras actividades, gestionar los pedidos sugeridos. La figura debajo representa el diagrama general de caso de uso de Thary:

**Fig. 3.** Caso de uso

Como se puede ver en el diagrama, los casos de uso identificados, se relacionan con los requisitos funcionales de la siguiente manera:

TABLA III
Trazabilidad entre casos de uso y requisitos funcionales

ID	Caso de uso	Actor principal	Requisitos relacionados
CU-01	Iniciar sesión	Usuario registrado	RF-09
CU-02	Cargar historial de consumo	Administrador	RF-01, RF-07
CU-03	Procesar archivo	Administrador	RF-02, RF-07
CU-04	Consultar demanda pronosticada	Personal de Inventario	RF-02, RF-05
CU-05	Consultar estado del inventario	Personal de Inventario	RF-04, RF-05
CU-06	Consultar recomendaciones de reposición y presupuesto	Personal de Inventario	RF-03, RF-05
CU-07	Registrar pedido	Personal de Inventario	RF-06
CU-08	Gestionar usuarios	Administrador	RF-08
CU-09	Consultar asistente virtual	Usuario registrado	RF-10

- Caso de uso CU-01 (Iniciar sesión (Usuario registrado)): Este caso de uso corresponde al RF-09, y le permite a cualquier usuario registrado ingresar al sistema mediante sus credenciales para acceder a las funcionalidades según su rol.
- Caso de uso CU-02 (Cargar historial de consumo (administrador)): Este caso de uso corresponde al RF-01, y permite al administrador seleccionar y cargar el archivo csv con el historial de consumo.
- Caso de uso CU-03 (Iniciar procesamiento(Administrador)): Corresponde al requerimiento RF-02, que hace referencia a que una vez cargado el archivo, el administrador debe de seleccionar la opción "Procesar archivo" para que el sistema inicialice el procesamiento de los datos.
- Caso de uso CU-04 (Consultar demanda pronosticada (Personal de inventario)): Este caso de uso corresponde al RF-02 y al RF-05, y le permite al usuario consultar la demanda mensual estimada para cada producto una vez el archivo ya ha sido procesado.
- Caso de uso CU-05 (Consultar estado del inventario (Personal de inventario)): Este caso de uso corresponde a RF-04, que permite al usuario visualizar tanto el inventario disponible como los productos que presentan condiciones de alerta.

- Caso de uso CU-06 (Consultar las recomendaciones de reposicion y presupuesto (Personal del inventario)): Corresponde al requisito RF-03, y le permite al usuario consultar las recomendaciones generadas por el sistema y la informacion asociada al presupuesto disponible
- Caso de uso CU-07 (Confirmar pedido (Personal de inventario)): Este se relaciona directamente con el RF-06, y permite marcar como completado las recomendaciones de pedido generadas por Thary. Los pedidos confirmados quedan registrados con el nombre de la persona que los completa.
- Caso de uso CU-08 (Gestionar cuentas de usuario(administrador)): Le permite al administrador el poder crear nuevas cuentas al personal del inventario
- Caso de uso CU-09 (Consultar asistente virtual (Usuario registrado)): Este caso de uso corresponde al RF-10, y le permite a cualquier usuario, sin importar su rol dentro del sistema, resolver dudas sobre el funcionamiento de Thary mediante un chatbot implementado con la API de Ollama.

Todos los casos de uso se documentan individualmente en la especificacion SRS presentada en el Anexo A, la cual contempla detalles como objetivo, actor principal, precondiciones y postcondiciones, flujo principales y alternativos.

Elementos como las precondiciones se establecen antes de iniciar una funcionalidad, mientras que las postcondiciones describen cual es el estado esperado del sistema al finalizar el proceso, es de esta manera que realizar una correcta especificacion de casos permite establecer una base a la hora de continuar con el proceso de diseño de arquitectura e implementacion de servicios.

3.2.4 Diseño de la Arquitectura

El sistema Thary fue diseñado basandose en una organizacion pipeline para el componente de prediccion y un estilo arquitectonico en capas de las cuales se distinguen tres principales:

- Capa de Presentacion: Esta capa es la reponsable de funcionalidades como carga de archivos, visualizacion de resultados, registro de pedidos y demas funcionalidades relacionadas con toda la interaccion que el usuario final va a tener con el sistema.
- Capa de Control: Esta capa es la encargada de recibir todas las solicitudes que el usaurio le haga a la aplicacion, y devolver los resultados esperados a la capa de presentacion

- **Capa de servicios de dominio:** En esta capa se encuentra el componente de predicción organizado como un pipeline secuencial compuesto por las diferentes etapas del sistema, en general, en esta capa se concentra toda la lógica de Thary. Dentro de esta misma capa también se ubica el módulo del asistente conversacional, el cual se comunica con un modelo de lenguaje ejecutado localmente a través de la API de Ollama, en lugar de depender de un servicio externo en la nube.

En cuanto a porque se decidió llevar la organización mediante un pipeline, esta decisión se tomó debido a que el proceso de predicción de demanda, por su naturaleza, corresponde a un flujo secuencial de transformación de los datos, por lo que el uso de esta estructura facilita que se pueda separar responsabilidades de una manera más sencilla y la trazabilidad de los resultados obtenidos.

Por otro lado, a la hora de elegir como se manejaría la persistencia de la información, se optó por simplemente utilizar archivos estructurados, en lugar de utilizar algún motor de base de datos relacional. Esta decisión se tomó considerando el alcance del proyecto y de esta manera, evitando involucrar complejidad adicional al mismo. Aun así, la arquitectura de Thary contempla el aislamiento de los datos para cada organización, de manera que todas cuentan con un espacio independiente evitando que puedan acceder a información de otro usuario.

Escenarios Arquitecturales: Para garantizar que Thary cuenta con una arquitectura que es capaz de responder a los atributos de calidad necesarios para un proyecto con sus características, se plantaron los siguientes escenarios:

- **Escenario 1 (Confiabilidad ante datos escasos):** Este escenario hace referencia a que cuando un usuario cargue el historial de un producto con menos de cuatro meses de registros, Thary no debe fallar ni excluir el producto, en su lugar, debe utilizar un método de estimación de respaldo, como la media móvil. Esto le aporta valor al sistema cuando no exista suficiente información histórica para usar un modelo más complejo.
- **Escenario 2 (Desempeño ante catálogos extensos):** Cuando se procesa un archivo con un número alto de productos diferentes, los cálculos de variables como puntos de reposición y cantidades sugeridas deben poder completarse de manera eficiente, sin que el tiempo de procesamiento se degrade de forma desproporcionada al tamaño del catálogo.
- **Escenario 3 (Confidencialidad de la información):** En este escenario se tiene en cuenta que Thary debe garantizar que los datos relacionados con el inventario y el historial de consumo de la organización, no se transmitan a ningún servicio externo del que Thary se encuentra alojado, incluyendo las consultas que un usuario le haga al asistente conversacional.

- Escenario 4 (Aislamiento entre organizaciones): Al trabajar con tantos usuarios/administradores, lo más probable es que en algún momento, al menos dos organizaciones distintas usen el sistema de manera simultánea, Thary debe asegurarse que ningún tipo de dato quede expuesto ni pueda ser modificado por otra organización
- Escenario 5 (Adaptabilidad del formato de datos): Este escenario se refiere a que cuando se cargue un archivo que pertenezca a un dominio diferente al caso de estudio principal, el cual fue salud, Thary tiene que ser capaz de reconocer y normalizar las columnas sin tener que modificar el código.

Estos escenarios permiten poder verificar que las decisiones arquitectónicas tomadas respondan a las necesidades que tiene el sistema, facilitando que posteriormente se pueda comprobar el funcionamiento de Thary

Análisis de Alternativas y Decisiones de Diseño (ADRs): Las decisiones de diseño que fueron adoptadas durante todo el desarrollo del sistema Thary y las alternativas consideradas.

- Decision 1 (Persistencia basada en archivos frente a base de datos relacional): Debido a que el sistema va a trabajar con información relacionada con los usuarios y los historiales de consumo que compartan con el sistema, se tuvo que seleccionar la manera más adecuada de almacenar dicha información. Teniendo en cuenta factores como el alcance del proyecto, se decidió que se utilizaran archivos estructurados independientes para cada la información de cada organización. La elección de esta alternativa se tomó con la intención de poder simplificar el despliegue y la administración del sistema, aunque cuenta con limitaciones como una menor capacidad para manejar consultas concurrentes a gran escala.
- Decision 2 (Selección del modelo de predicción por cada): Existen distintos productos del inventario que pueden presentar patrones de demanda diferentes entre sí, teniendo esto en cuenta, al inicio del desarrollo se propusieron dos alternativas, la primera era generalizar para todo el sistema el uso de un mismo modelo, y la segunda implementar tres modelos distintos. Se optó que la mejor opción es entrenar diferentes modelos candidatos (XGBoost, regresión lineal, y media móvil), comparar los resultados de los 3 y dependiendo de si el consumo es estable, esporádico o muy variable, seleccionar el que presente el menor error de predicción
- Decision 3 (Asistente conversacional o un servicio externo): Puede que los usuarios del sistema tengan dudas sobre el funcionamiento de Thary, pero que no quieran tener que salir del sistema para consultar cómo funciona. Teniendo esto en cuenta, se plantearon dos

alternativas, la primera consistía en implementar un servicio de inteligencia artificial en la nube, o implementar un chatbot ejecutado localmente mediante la api de Ollama. Al final, se optó por esta última, debido a factores como de presupuesto, seguridad, ya que así nos aseguramos que ninguna consulta realizada por el usuario con información sensible tenga que salir del entorno donde está alojado el sistema, aunque es probable que la calidad de las respuestas del asistente dependen del modelo local disponible, y su disponibilidad depende de que el servicio Ollama se encuentre corriendo sin problemas.

- **Decision 4 (Validación temporal y validación cruzada):** A la hora de validar los resultados de Thary, los cuales son conseguidos a partir de series de tiempo (historial de consumo mensual), se pensó en dos propuestas, la primera se basaba en hacer una validación cruzada aleatoria, que supone hacer una partición sin considerar el orden cronológico, mientras que la segunda se basaba en una validación cruzada temporal de origen móvil, en la que se generan varias particiones de entrenamiento y pruebas consecutivas. Se decidió optar por la segunda opción,

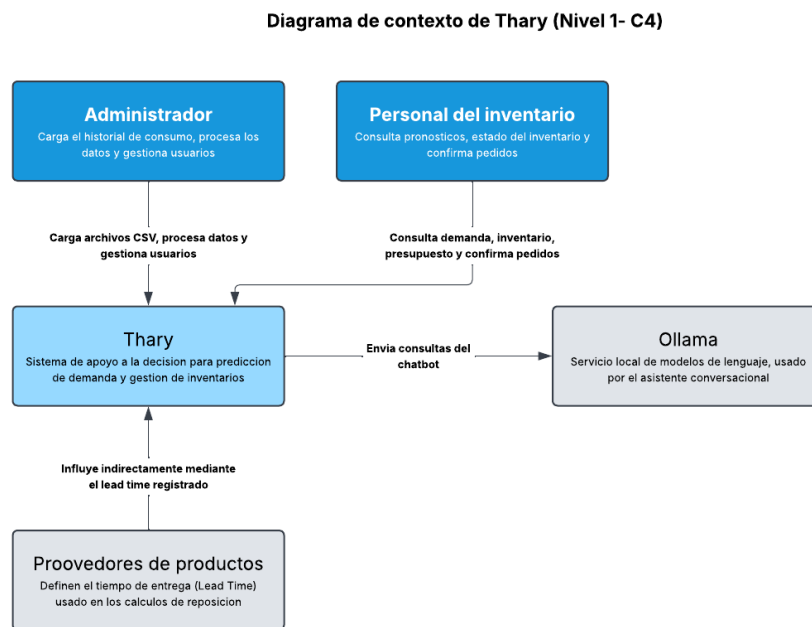


Fig. 4. Diagrama de contexto del sistema (nivel 1 del modelo C4).

Representación de la Arquitectura (Modelo C4): Como se observa en la Fig. 4, el sistema interactúa con los actores identificados en la sección 3.2.1 y como se implementó el uso de Ollama como asistente conversacional.

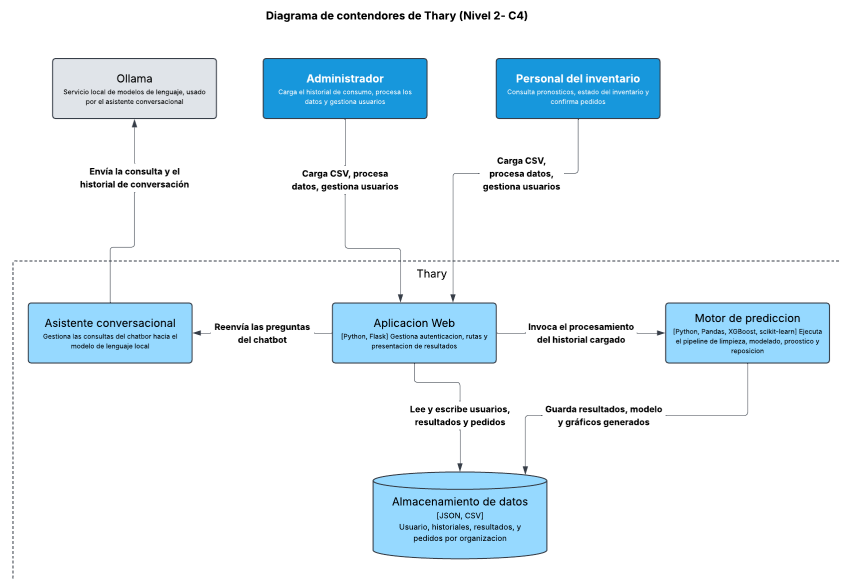


Fig. 5. Diagrama de contenedores del sistema (nivel 2 del modelo C4).

En la figura 5 se puede observar como se organizaron los contenedores del sistema Thary, factores como la aplicación web, el motor de predicción, el almacenamiento de datos y el asistente conversacional son contenedores independientes que se comunican entre sí para el correcto funcionamiento del sistema.

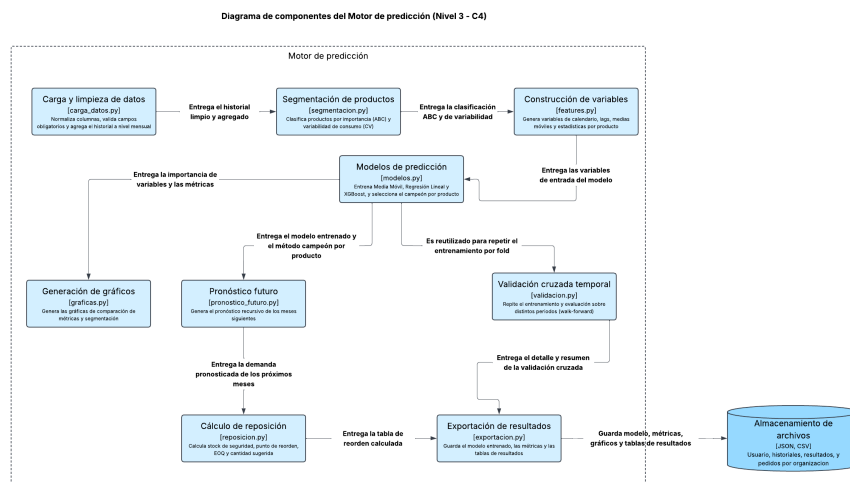


Fig. 6. Diagrama de componentes del sistema (nivel 3 del modelo C4).

Finalmente, el diagrama de componentes detalla a profundidad el funcionamiento interno del motor de predicciones.

3.2.5 Diseño Detallado

Modelo de Datos: Conceptualmente, el modelo de datos del sistema Thary utiliza un modelo relacional, en el que la información se organiza en 12 tablas relacionadas entre sí, definidas mediante claves foráneas: empresas, usuarios, pedidos, actividad (el historial de movimientos), y corridas. Esta última es la tabla principal que representa cada vez que un usuario (administrador) sube un CSV, junto con 7 tablas que vienen de cada una de las corridas: `segmentacion`, `metricas_modelos`, `predicciones_backtest`, `pronostico_futuro`, `reorder`, `validacion_cruzada_folds`, `validacion_cruzada_resumen`. Cada archivo que se sube crea una fila nueva en corridas, aunque no se borra el historial, la app solo muestra la corrida más reciente.

Lógicamente y físicamente, el modelo hace uso de MySQL (motor de base de datos relacional), el sistema se conecta a este usando SQLAlchemy. En la tabla `empresas`, se almacena el ID y el nombre de cada organización que es registrada en el sistema, por otra parte, en la tabla `usuarios` se guarda la información del usuario, como su nombre, correo electrónico, contraseña (cifrada), el rol, ya sea admin o empleado, y el ID de la empresa a la que pertenece. La tabla `pedido` también se almacena de forma independiente en el archivo `pedidos.json`, este archivo está separado de los resultados del procesamiento y es único para cada empresa, así se garantiza que cuando se cargue un nuevo historial, no se elimine el registro de los pedidos ya confirmados. Por otro lado, las entidades de registro de consumo histórico, recomendación de reposición y comportamiento del producto son generadas después de que el sistema procese el archivo CSV que fue cargado por el administrador. Estos resultados se guardan en los archivos `segmentacion_productos.csv` y `reorder_table.csv`, en una carpeta de resultados distinta para cada empresa.

No toda la información se queda en la base de datos, ya que los archivos CSV que el usuario sube, las gráficas generadas y el modelo XGBoost entrenado se siguen almacenando como archivos en disco, los cuales están organizados en carpetas por empresa. Solo se traslada a MySQL la información que pueda estructurarse en filas y columnas.

TABLA IV
Entidades principales del modelo de datos.

Entidad	Descripción	Cardinalidad
Empresa	Organización que utiliza el sistema, identificada de forma única.	1 : N
Usuario	Persona registrada en el sistema, con un rol (administrador o personal de inventario), asociada a una empresa mediante clave foránea.	1 : N
Pedido	Proveedor, cantidad, costo y usuario responsable de un pedido confirmado, asociado a una empresa.	1 : N
Corrida	Registro de un procesamiento de archivo, con sus parámetros y métricas globales; tabla padre de los resultados detallados del pipeline.	1 : N
Resultados de la corrida	Conjunto de tablas hijas de Corrida: segmentación, métricas por modelo, predicciones de backtest, pronóstico futuro, reposición y validación cruzada.	N : 1 (respecto a Corrida)
Actividad	Bitácora de acciones realizadas por los usuarios (subir archivo, confirmar/deshacer pedido, crear usuario), asociada a una empresa.	1 : N

La Tabla IV resume las entidades del modelo con el detalle de cada campo y su cardinalidad

```
ETIQUETAS_COLUMNS_OBLIGATORIAS = {
    "fecha": "Fecha (o Date, Sale_Date)",
    "producto_id": "Producto (o Product, Product_ID)",
    "demanda": "Demanda (o Demand, Sales, Quantity_Sold)",
}
```

```
def cargar_y_limpiar(ruta_archivo):
    try:
        df = pd.read_csv(ruta_archivo)
```

```

except pd.errors.EmptyDataError:
    raise ValueError("El archivo está vacío. Sube un CSV con al men
except pd.errors.ParserError:
    raise ValueError(
        "No se pudo leer el archivo como CSV. Verifica que sea un a
        "separado por comas (por ejemplo, no un Excel renombrado a
    )
except UnicodeDecodeError:
    raise ValueError(
        "El archivo tiene una codificación de caracteres que el sis
        "Guárdalo como CSV con codificación UTF-8 e intenta de nuev
    )

if df.empty:
    raise ValueError("El archivo no tiene filas de datos, solo encab

df = df.rename(columns={
    "Sale_Date": "fecha", "Date": "fecha", "date": "fecha", "Fecha
    "Product_ID": "producto_id", "Product": "producto_id", "Produc
    "Quantity_Sold": "demanda", "Sales": "demanda", "Demand": "dem
# ... variantes adicionales para precio, inventario, categoria, provee
})

columnas_faltantes = [c for c in ETIQUETAS_COLUMNAS_OBLIGATORIAS i
if columnas_faltantes:
    faltan = ", ".join(ETIQUETAS_COLUMNAS_OBLIGATORIAS[c] for c in
    raise ValueError(
        f"Al archivo le falta al menos una columna obligatoria: {f
        "Agrégala al CSV (con ese nombre exacto u otro de los acep
    )

```

En esta sección del archivo (`services/prediccion/carga_datos.py`), se observa la etapa de carga y limpieza de los datos. Para garantizar que el sistema pueda adaptarse y funcionar para todo tipo de dominio de inventarios, se normalizan los nombres de las columnas de los archivos CSV con el fin de mantener un esquema más heterogéneo, de manera que el sistema pueda aceptar archivos de distintos dominios sin tener la necesidad de modificar el resto del pipeline. Para facilitarle al usuario el proceso de cargar un archivo, se capturan los errores mas

comunes para el sistema a la hora de leer un CSV y se muestran a manera de mensaje de error en un lenguaje sencillo y facil de entender.

Otra practica que fue aplicada es el manejo de errores con mensajes dirigidos al usuario final, de esta manera, se evita que se propaguen los errores internos del sistema y el usuario sabe que es lo que esta ocurriendo y que es lo que se debe hacer para resolverlo.

Diseño UX/UI: Para realizar el diseño de interaccion de Thary, se decidio separar las pantallas en dos: Una dirigida exclusivamente a los administradores y otra dirigida a los empleados del inventario. Al ingresar al sistema, un usuario con el rol de administrador sigue el siguiente flujo: El administrador inicia en la pantalla de carga de archivo (*index*), al haber cargado el archivo correctamnete y que este sea valido, se le redirige automaticamente al dashboard con los resultados. En el sidebar, ademas de encontrar demas opciones como estado del inventario, presupuesto y pedidos, el administrador puede navegar hacia la gestion de usuarios, donde puede crear cuentas nuevas para su personal de inventario.

El flujo de los empleados es bastante similar, con la diferencia de que el perfil de un empleado cuenta con menos privilegios que el perfil de un administrador, por lo que al iniciar sesion, el empleado inicia directamente en el dashboard de la demanda pronosticada en base al csv que ya anteriormente fue cargado al sistema por el administrador que creo su cuenta, ya desde ahi, puede navegar a las mismas funcionalidades del analisis de los datos que el administrador: estado del inventario, presupuesto y pedidos.

Una vez finalizadas sus tareas, los usuarios pueden cerrar sesion en su cuenta.

Los criterios de usabilidad que fueron tomados en cuenta fueron:

- Uso de alertas visuales codificadas por color para indicar facilmente que productos se encuentran en o por debajo de su punto de reposicion y requieren ser reabastecidos inmediatamente.
- Se estandarizo el uso del formato de la moneda colombiana en la seccion de presupuestos y montos monetarios.
- Cuando un usuario con el rol de empleado inicie sesion, pero su respectivo administrador no subio ningun archivo csv, en ves de ver la pantalla del dashboard, vera una pantalla con el mensaje de que una vez el admin cargue el archivo, podra ver los analisis.
- Se incorpore un boton de acceso a un asistente conversacional que despliega una ventana

emergente de chat en la que podrá resolver sus dudas sobre el funcionamiento del sistema y el análisis de los datos.

En cuanto a criterios de accesibilidad, se tomaron en cuenta los criterios de usabilidad, pero no se tomaron en cuenta mecanismos adicionales para hacer mas accesible el sistema como algun tipo de lector de pantalla para personas con discapacidad visual, o implementar un sistema de navegacion completa por teclado para alguna persona que no pueda usar un mouse. Estas falencias se reconocen como una limitacion del diseño actual, y se espera mejorarlas en el futuro.

3.2.6 Análisis de Amenazas y Diseño Seguro

Debido a su naturaleza, Thary es un sistema que maneja informacion que podria considerarse potencialmente sensible, como los datos de los usuarios, de consumo, entre otros. Teniendo esto en cuenta, se realizo un analisis en el que se identificaron las amenazas siguiendo el modelado de amenazas STRIDE:

- **Suplantación de identidad:** mitigada mediante autenticación basada en usuario y contraseña, almacenando esta última mediante un algoritmo de hash (*scrypt*) y nunca en texto plano.
- **Manipulación de datos:** Para evitar la manipulacion de los datos, se garantiza que la cantidad y el costo de un pedido confirmado se tome siempre del análisis generado por el sistema, y no de valores que el usuario pueda modificar en el formulario de confirmación.
- **Repudio:** Aunque Thary no cuenta con un sistema en el que se registre quien inicio sesion y cuando, a la hora de confirmar un pedido, en `pedidos.json` se registra el nombre del usuario responsable, y la fecha y hora de confirmacion, lo que motoga el riesgo a la hora de confirmar los pedidos, pero no para demas acciones del sistema com la carga de un archivo y la creacion de un usaurio.
- **Divulgación de información:** Se evita que exista divulgacion de informacion mediante el aislamiento de los datos por empresa, de forma que ninguna organización pueda acceder a los datos de otra.
- **Denegacion de servicio:**
- **Elevación de privilegios:** Para evitar este riesgo, se implemento un control de acceso basado en roles, de forma que las funciones de carga de archivos, procesamiento y consulta del panel técnico estén restringidas exclusivamente al perfil administrador.

3.3 Construcción, Integración y Despliegue

Thary es un sistema que se desarrollo mediante el uso de las siguientes tecnologías: Python, Flask como framework, librerías como scikit-learn, Pandas, y XGBoost.

A la hora de ejecutar el sistema en Flask, se tiene que ejecutar el comando `run.py` en un entorno virtual (*venv*), todas las dependencias necesarias para la correcta ejecucion del proyecto se encuentran resumidas en el archivo `requirements.txt` para su facil instalacion.

Por su parte, el control de versiones del sistema se llevo a cabo mediante una seria de commits en GitHub en el que se documentaron los cambios mas significativos del sistema durante todo el tiempo de desarrollo.

Construcción: La principal practica de codificacion que se siguio durante el desarrollo e implementacion del sistema Thary es la separacion de responsabilidades, practica la cual se vio reflejada en la organizacion de archivos y carpetas del proyecto:

```
sistema_prediccion_inventario/
  app/
    routes/
      auth.py
      main.py
    services/
      auth.py
      db.py
      pedidos.py           # Registro de pedidos confirmados
      actividad.py         # Acciones de usuario
      prediccion/          # pipeline
        carga_datos.py     # 1. Carga y limpieza
        segmentacion.py    # 2. Segmentacion ABC/variabilidad
        features.py        # 3. Construcción de variables
        modelos.py         # 4. Entrenamiento y seleccion
        regresion_lineal.py# 4.1. Modelo de Regresion Lineal
        validacion.py      # 5. Validacion cruzada temporal
        pronostico_futuro.py# 6. Pronostico recursivo
        reposicion.py      # 7. Puntos de reorden y EOQ
        graficas.py        # 8. Generacion de graficos
```

```
        exportacion.py      # 9. Exportacion de resultados
templates/
tests/                        # pytest
    test_carga_datos.py
    test_modelos.py
    test_reposicion.py
```

Como se puede observar, la organizacion se divide en tres capas: `routes/` con el control del sistema, `services/` con la logica de negocio y, dentro de esta ultima, `prediccion/`. Es en esa ultima capa, en la cual cada una de las 9 etapas del pipeline se encuentran alojadas en nueve modulos independientes: carga de datos, segmentacion, construccion de variables, modelos, validacion, pronostico, reposicion, graficas y exportacion. La principal ventaja de esta organizacion, es que permite que se pueda modificar una etapa especifica del flujo sin afectar directamente a las demas partes, ademas de facilitar el mantenimiento y un entendimiento mas facil del codigo.

Como complemento a esta organizacion, se incorporaron pruebas automatizadas escritas con *pytest*, las cuales fueron agrupadas en tres archivos dentro de la carpeta `tests/`:

- `test_carga_datos.py`, en este archivo se verifican los mensajes de error ante archivos invalidos y el reconocimiento de columnas heterogeneas
- `test_modelos.py`, en el que se verifica la logica de seleccion del modelo campeon por producto
- y por ultimo, `test_reposicion.py`, en donde se verifican los calculos de stock de seguridad, punto de reorden y EOQ

(Los resultados obtenidos al ejecutar esta suite de pruebas se presentan en el Capitulo 4.)

Integración: Las rutas de la aplicacion se organizan en el archivo `app/__init__.py`, mediante el mecanismo de Blueprints de Flask, el cual permite que se pueda agrupar un conjunto de vistas en un modulo independiente y registrarlo en la aplicacion principal. Thary cuenta con dos blueprints: `main_bp` (dashboard, inventario, presupuesto, pedidos y panel tecnico) y `auth_bp` (inicio de sesion, registro y gestion de usuarios), que se registran de la siguiente manera:

```
app.register_blueprint(main_bp)
app.register_blueprint(auth_bp)
```

Esta organizacion permite que cada blueprint se desarrolle y se pruebe de forma independiente, y que agregar un nuevo conjunto de rutas al sistema no implique tener que modificar el codigo de los blueprints que ya existen.

Por otra parte, para integrar correctamente el modelo de prediccion con la aplicacion web, en el archivo `services/sistema_prediccion.py` se incluyo una funcion de entrada, `ejecutar_sistema()`, la cual se encarga de orquestrar secuencialmente las nueve etapas del pipeline y devolver un resultado. Esta decision de integracion se tomo tomando en cuenta la separacion de capas ya establecida para todo el proyecto. En este caso, la capa de servicios de dominio se mantiene independiente de la de presentacion.

```
def ejecutar_sistema(
    ruta_archivo, carpeta_resultados=None, presupuesto_capital_trabajo
):
    # 1. Carga, limpieza y agregacion mensual
    df = cargar_y_limpiar(ruta_archivo)
    df_mensual = agregar_mensual(df)

    # 2. Segmentacion de productos
    producto_comportamiento = calcular_comportamiento_producto(df, df_

    # 3. Feature engineering
    df_mensual, df_model, feature_cols = construir_features(df_mensual

    # 4. Modelos
    modelo_lr, pred_lr, features_lr = entrenar_regresion_lineal(X_train
    model, pred_xgb = entrenar_xgboost(X_train, y_train_log, X_test)
    pred_baseline_3 = prediccion_baseline_movil(test_df)

    # 5. Validacion cruzada temporal
    detalle_validacion_cruzada, resumen_validacion_cruzada = validacio

    # 6. Pronostico recursivo de los proximos N meses
    pronostico_futuro_df, ... = generar_pronostico_futuro(...)

    # 7. Puntos de reorden y EOQ
    reorder_df, ... = calcular_tabla_reorden(...)
```

```
# 8. Graficas y exportacion de artefactos
...

return { ... }
```

Otro punto de integracion relevante es que Thary tiene dos maneras de almacenar la informacion: la base de datos relacional (MySQL), para datos que sean estructurados como numeros o textos cortos y el sistema de archivos para archivos que por su naturaleza no se podria poner en una tabla, en el caso de Thary, estos archivos serian el archivo del modelo XGBoost ya entrenado, y las imagenes de las graficas. Cada vez que se procesa un CSV, en MySQL se agrega una fila nueva, nunca se borra nada, pero los archivos, es decir, las graficas, modelo entrenado, etc, van siempre a la misma carpeta de la empresa a la que pertenezcan, y cada vez que se procesa un nuevo archivo, se sobrescriben los anteriores. Asi que, si se hace una serie de corridas, en MySQL habra varias filas completas, pero en la carpeta, solo existiran los archivos de la ultima corrida.

app/routes/main.py:

```
def carpeta_resultados(empresa_id):
    """Carpeta donde ejecutar_sistema escribe el pronostico, los
    graficos y demas archivos de resultado de una empresa puntual."""
    ruta = os.path.join(current_app.config["RESULTADOS_FOLDER_BASE"], e
    os.makedirs(ruta, exist_ok=True)
    return ruta
```

La ruta de la carpeta depende del id de la empresa (empresa_id), y no del id de la corrida (corrida_id), por lo que, la carpeta siempre es la misma para una misma empresa, sin importar cuantas veces se haya procesado un archivo.

Por otra parte, tal como se explico en capitulos anteriores, se definio un control de acceso basado en roles, esto con el objetivo de poder diferenciar a que funcionalidades debe tener acceso un usuario dependiendo de su rol. El administrador (admin) debe ser el unico usuario que cuente con todos los privilegios de acceso a la informacion sensible de la organizacion, mientras que para otros usuarios con menor rango, (empleado), convendria restringir este acceso completo a solo aquellas funcionalidades menos invasivas que sean necesarias para el desempeño de su trabajo, como todas aquellas que describen el analisis de los datos y la lista de pedidos:

app/services/auth.py:

```
def admin_required(vista):
    @wraps(vista)
    def envoltura(*args, **kwargs):
        if "username" not in session:
            flash("Inicia sesión para continuar.")
            return redirect(url_for("auth.login"))
        if session.get("rol") != "admin":
            flash("Esa sección es solo para el administrador.")
            return redirect(url_for("main.dashboard"))
        return vista(*args, **kwargs)
    return envoltura
```

Integración continua (CI) El proyecto utilizo GitHub Actions para la automatización de pruebas unitarias (.github/workflows/tests.yml). Este se ejecuta automáticamente cada vez que se haga un *push* o *pull request* en el repositorio. El archivo cuenta con una configuración que permite instalar tanto Python como todas las dependencias necesarias, y ejecuta las pruebas unitarias con pytest:

```
name: Tests

on:
  push:
  pull_request:

jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: "pip"
```

- **run:** `pip install -r requirements.txt`
- **run:** `pytest tests/ -q`

El flujo solo cubre la parte de CI (Integración Continua), en cuanto a CD (Despliegue Continuo), se decidió hacerlo de manera manual, principalmente porque al ser un proyecto desarrollado por una sola persona, es decir, sin múltiples entornos ni un equipo que dependa de constantes despliegues, el beneficio de un pipeline de CD completo es menor frente a la complejidad adicional que implicaría configurarlo.

Despliegue: El despliegue del sistema Thary (además del entorno de desarrollo local), se encuentra en la nube mediante la plataforma Railway.

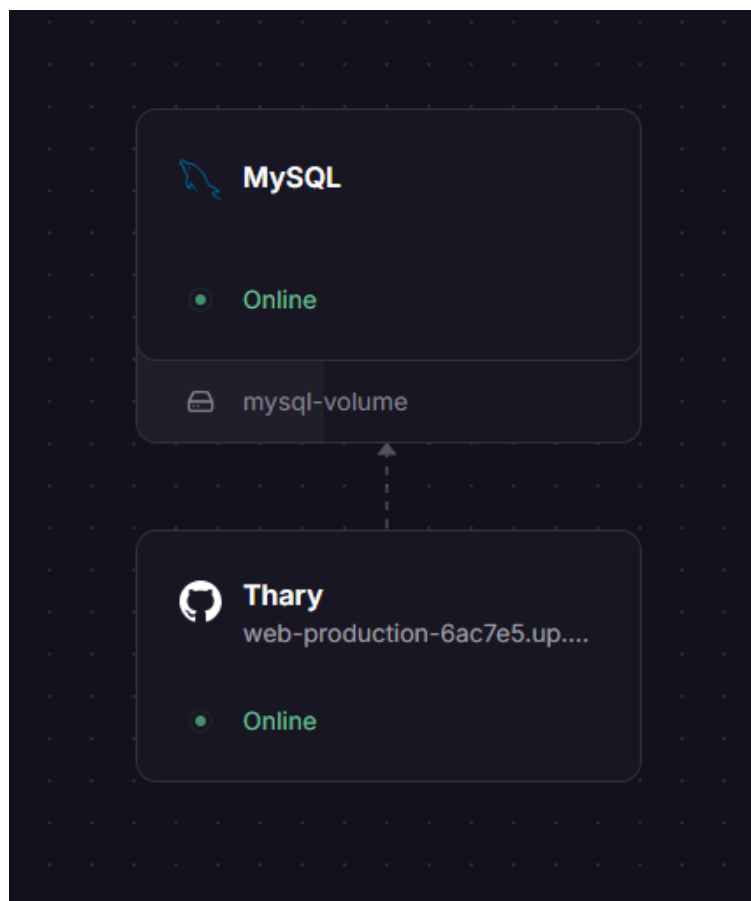


Fig. 7. Servicios desplegados en Railway: aplicación web (Thary) y base de datos MySQL.

Como se ve en la figura, Thary y la base de datos, se encuentran corriendo en Railway

como servicios independientes, los cuales se comunican entre si para compartir información y funcionalidades.

La aplicacion se ejecuta mediante un servidor WSGI de produccion (Gunicorn), configurado a traves de un archivo `Procfile`:

```
web: gunicorn run:app --bind 0.0.0.0:$PORT
```

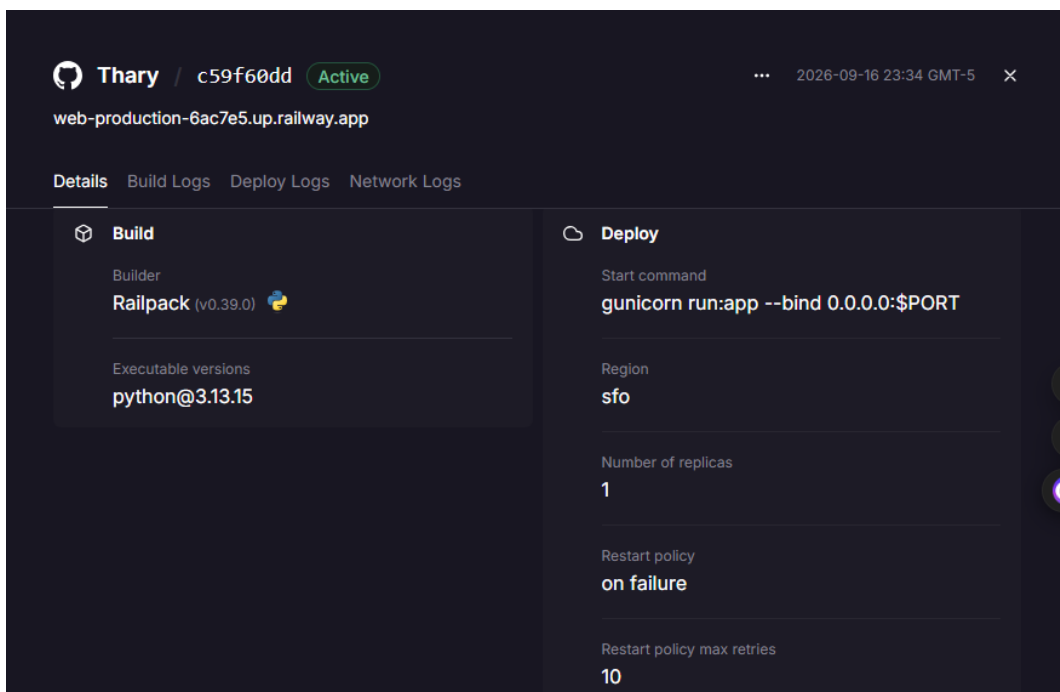


Fig. 8. Detalles de construcción y despliegue del servicio Thary en Railway.

Railway detecta y se encarga de ejecutar este comando *start command* cuando se realiza un despliegue. Automaticamente identifica la version de python con la que se esta trabajando, en este caso, `.python-version (3.13)`. El proyecto fija todas sus dependencias en `requirements.txt`, incluyendo las versiones exactas de las librerias de prediccion (pandas, scikit-learn, XGBoost, statsmodels) y las necesarias para la conexion a la base de datos (SQLAlchemy, PyMySQL) y el servidor de produccion (gunicorn).

En cuanto a la persistencia de los datos, esta se manejó mediante MySQL. La configuracion de conexion a la base de datos solo se obtiene de las variables de entorno:

```
def _config() :
```

```
return {  
  "host": os.environ.get("DB_HOST", "127.0.0.1"),  
  "port": os.environ.get("DB_PORT", "3307"),  
  "user": os.environ.get("DB_USER", "thary_app"),  
  "password": os.environ.get("DB_PASSWORD", ""),  
  "database": os.environ.get("DB_NAME", "sistema_inventario"),  
}
```

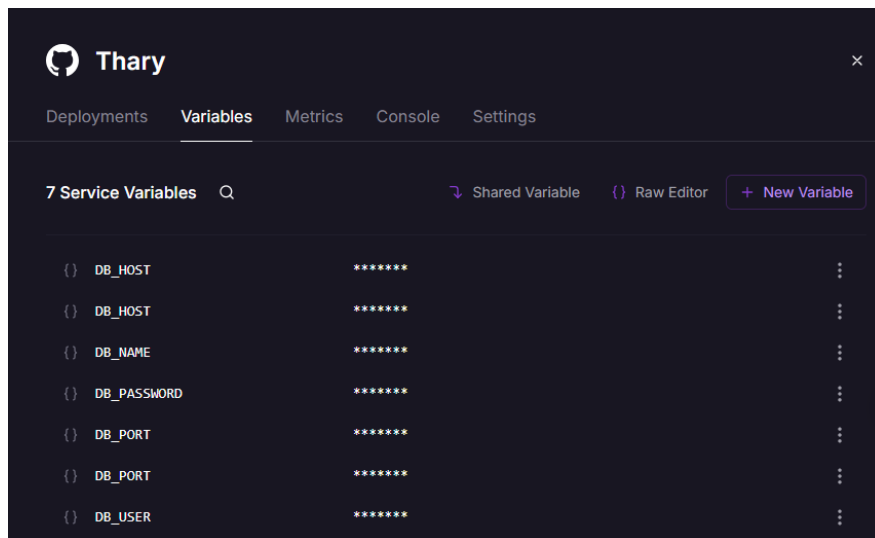


Fig. 9. Variables de entorno del servicio Thary en Railway, con los valores ocultos.

Las credenciales se configuran en el panel de Railway como variables de entorno del servicio, nunca visibles en el repositorio de código.

El servicio de Thary no cuenta con un volumen configurado, esto puede verse en el panel de administración en el que no se ve la pestaña de "volúmenes". Esto quiere decir que los archivos que están en el servicio son efímeros, es decir, cada vez que se redesplice el servicio, como por ejemplo, por una actualización del código o un reinicio del servicio, se crea un nuevo contenedor, por lo que se pierden por completo los archivos generados por el sistema en las corridas anteriores del pipeline. Esto se reconoce como una limitación del entorno de producción.

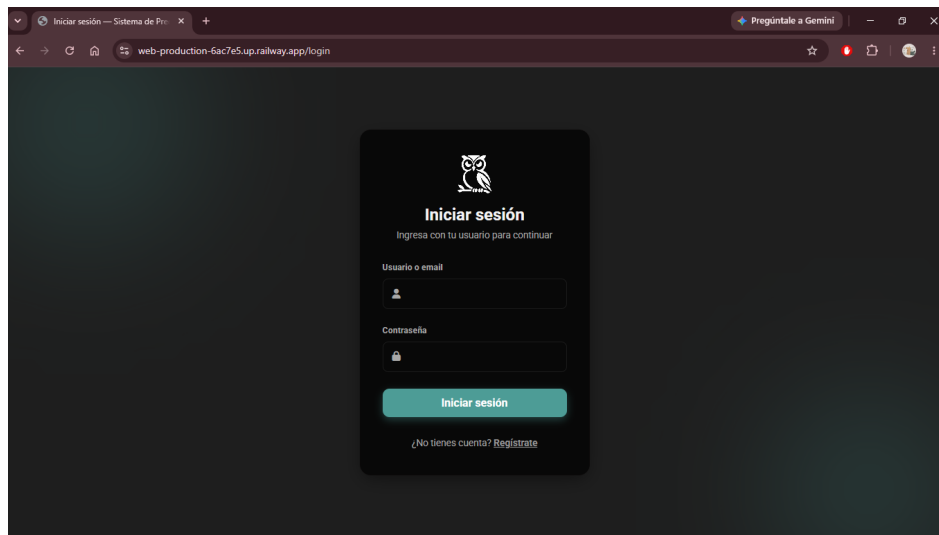


Fig. 10. Pantalla de inicio de sesión de Thary, accedida desde su URL pública en Railway.

Finalmente, la figura muestra la aplicación funcionando correctamente en el entorno de producción a través de la URL pública asignada por Railway.

3.4 Plan de Verificación y Validación

Estrategia de verificación:

Estrategia de validación del modelo de predicción:

Criterios de aceptación:

Plan de validación con usuarios:

4 Capítulo 4: Análisis de Resultados y Juicio de Ingeniería

4.1 Resultados de Verificación (Pruebas de Software)

La verificación del sistema de Thary, se llevó a cabo haciendo pruebas automatizadas escritas con *pytest*. Las pruebas se enfocaron en evaluar tres componentes principales

4.2 Resultados de Validación (Evaluación del Usuario)

4.2.1 Validación del modelo de predicción

A continuación, se presentan los resultados obtenidos en la validación del modelo de predicción. Estos resultados se obtuvieron luego de aplicar la estrategia de validación ya definida anteriormente, sobre el historial real de consumo de 856 productos (medicamentos) del año 2021 - 2022 en la Clínica Nuestra Señora de los remedios.

Las métricas obtenidas al evaluar los tres modelos candidatos, junto con el método de selección de modelo por producto. Todo esto sobre el último periodo de prueba disponible, que corresponde a junio 2022, con seis meses de entrenamiento previo.

TABLA V
Métricas del holdout final por modelo.

Modelo	MAE	RMSE	MAPE (%)	WAPE (%)
Media Móvil (3 meses)	48.79	243.65	80.32	23.86
Regresión Lineal	56.43	276.41	95.30	27.60
XGBoost (log1p)	50.59	254.82	65.12	24.74
Sistema final (selección por producto)	39.60	207.14	57.63	19.37

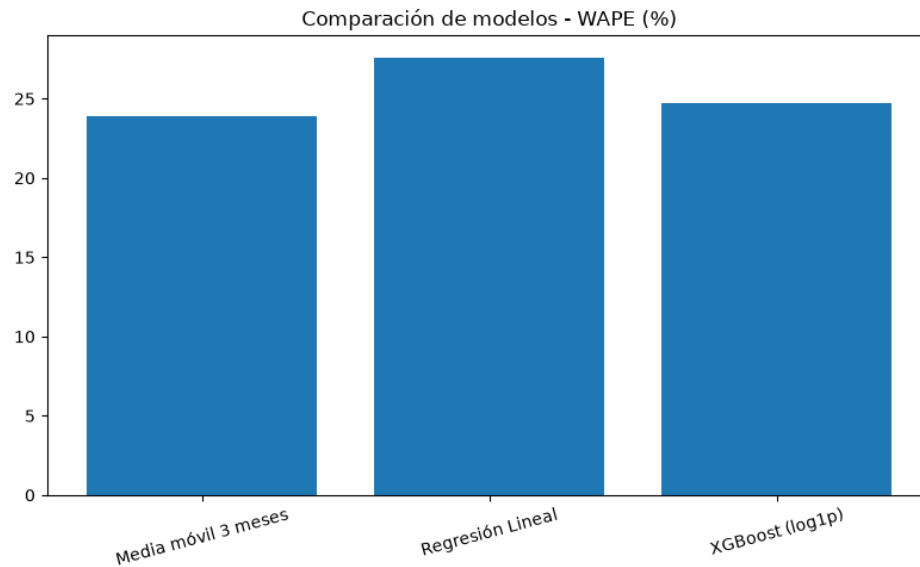


Fig. 11. Comparación de WAPE entre los tres modelos candidatos (holdout final).

4.2.2 Consistencia mediante validación cruzada temporal

Después de obtener los resultados anteriores, se propuso el uso de una validación cruzada para descartar que el resultado obtenido dependiera de las condiciones y particularidades de un único periodo de prueba.

La idea principal, es probar el modelo varias veces simulando estar en un mes distinto al pasado, para poder evaluar si acierta de forma consistente y no solo una vez por casualidad. Para poder predecir un mes, solo se puede usar los meses que vinieron antes, no se pueden usar los datos que serían "Futuros", porque queremos simular un escenario real, y en ese caso, no se tendría esa información.

1. Prueba 1 (Fold 1): Es el más antiguo, se le enseña al modelo con los primeros 4 meses del año (Enero, febrero, Marzo, Abril), y se hace que el sistema adivine Mayo, Junio y Julio. Se compara lo que adivino contra los resultados reales de esos meses y se registra que tan grande fue el error.
2. Prueba 2 (Fold 2): En esta segunda prueba o Fold, se le enseña al modelo con 5 meses en vez de 4, es decir, de enero a Mayo, y se le pide que prediga Junio, Julio y agosto, una vez más, se compara y se registra el error.
3. Prueba 3 (Fold 3): Por último, se le enseña al modelo con 6 meses de Enero a junio y se

le solicita al sistema que prediga julio , agosto y septiembre, se compara y, una vez mas, se registra el error.

Dicha validacion cruzada temporal se implemento en el sistema en el archivo `app/services/prediccion/validacion.py`.

```
TEST_SIZE_MESES = 3
MIN_TRAIN_MESES = 3
MAX_FOLDS = 3
```

```
def _armar_folds(meses_ordenados):
    n = len(meses_ordenados)
    folds = []
    fin_test = n
    while len(folds) < MAX_FOLDS:
        inicio_test = fin_test - TEST_SIZE_MESES
        if inicio_test < MIN_TRAIN_MESES:
            break
        folds.append((meses_ordenados[:inicio_test], meses_ordenados[inicio_test:fin_test]))
        fin_test -= 1
    folds.reverse()
    return folds
```

Se definen tres constantes, `TEST_SIZE_MESES`, `MIN_TRAIN_MESES` y `MAX_FOLDS`, que definen el tamaño de la ventana de prueba, el tamaño minimo de la ventana de entrenamiento y el numero maximo de folds a generar, respectivamente.

Se arma la funcion que genera las particiones `_armar_folds`, esta recibe una lista de todos los meses disponibles organizados cronologicamente. Se define el fold mas reciente, es decir, los ultimos meses disponibles `TEST_SIZE_MESES`. En cada iteracion se retrocede el punto de corte de un mes hacia atras en el tiempo, de esta manera, se crea un nuevo Fold cuyo entrenamiento crece en un mes respecto al anterior. Este proceso se repite hasta completar `MAX_FOLDS`, o hasta que ya no quede suficiente historial y el tamaño de la ventana de entrenamiento sea menor a `MIN_TRAIN_MESES` meses de entrenamiento.

Al finalizar, los folds se ordenan en orden cronológico del más antiguo al más nuevo antes de ser devueltos.

Al ya tener los folds organizados, el sistema entrena nuevamente los tres modelos candidatos, y selecciona el mejor modelo por producto, evaluando el resultado bajo la etiqueta de "Sistema final". Es decir, es la estrategia que usa Thary para predecir el consumo, en vez de escoger uno solo, para cada producto, el sistema entrena los 3 modelos y selecciona el que se equivoca menos con ese producto en particular. El proceso se repite producto por producto.

Un aspecto importante es que, la elección de que modelo usar para cada producto se hace solo con los datos de entrenamiento, antes de ver los datos de prueba. Esto demuestra que no se está "haciendo trampa, ya que el sistema no tiene acceso a los datos de prueba, y por lo tanto, no puede "aprender" de ellos.

Aun así, solo esto no basta para confiar en el resultado, ya que aun si el sistema está eligiendo el modelo de forma honesta, un único experimento podría haber salido bien por casualidad y no porque el sistema de verdad sea bueno.

Es por eso que la validación cruzada le aporta valor al sistema, ya que al repetir este mismo proceso (entrenamiento, elección del modelo y evaluación) en diferentes periodos de prueba, se confirma que el resultado obtenido en el holdout final no fue un resultado aislado, sino que se repite de forma consistente en distintos periodos de prueba.

En la siguiente tabla se describen el promedio y la desviación estándar de las métricas obtenidas en cada fold, para cada modelo candidato y el sistema final:

TABLA VI
Resumen de la validación cruzada temporal (promedio $\pm$ desviación, 3 folds).

Modelo	MAE	RMSE	MAPE (%)	WAPE (%)
Media Móvil	44.27 $\pm$ 3.70	200.82 $\pm$ 41.31	82.38 $\pm$ 1.85	22.31 $\pm$ 1.29
Regresión Lineal	58.13 $\pm$ 2.21	270.35 $\pm$ 6.32	106.24 $\pm$ 10.64	29.38 $\pm$ 1.84
XGBoost	46.58 $\pm$ 3.46	223.64 $\pm$ 31.40	64.93 $\pm$ 2.25	23.48 $\pm$ 1.15
Sistema final	36.25 $\pm$ 2.54	174.24 $\pm$ 27.89	59.13 $\pm$ 1.84	18.27 $\pm$ 0.81

De la tabla se puede observar que:

De la tabla se puede observar que:

- El sistema final es el que obtiene el mejor promedio en las cuatro metricas evaluadas, y tambien la menor desviacion estandar en todas ellas. Esto es significativo, porque demuestra que vale la pena la complejidad de elegir un modelo distinto para cada producto: el sistema final no solo es mas preciso en promedio, sino tambien mas consistente y estable entre los distintos periodos evaluados.
- La media movil es el segundo mejor modelo, seguido por XGBoost y finalmente la regresion lineal. Cuando se comparan los tres modelos candidatos de forma individual, la media movil es la que obtiene un mejor MAE, RMSE y WAPE, mientras que solo XGBoost la supera en MAPE. El hecho de que ninguno de los dos modelos mas sofisticados domine de forma general sobre la linea base avala la decision de seleccionar un modelo distinto por producto, ya que no existe un unico modelo que sea el mejor en todos los casos.
- La Regresion Lineal es la que tiene los peores resultados en las cuatro metricas, ademas de que su MAPE es el que mas varia entre los 3 folds. Esto es coherente con la tendencia de este modelo a sobreestimar los productos de alta demanda, observada previamente en el analisis de dispersion entre modelos, lo que ayuda a explicar por que su error resulta mas alto que el de los otros dos modelos.
- Finalmente, se observa que la efectividad del sistema final no se debe solamente a "elegir el mejor modelo en general", que en este caso seria la media movil, sino a elegir un modelo distinto para cada producto. En caso de haber elegido media movil para todos los productos, el error hubiera sido de 22.31% de WAPE, mientras que al elegir el mejor modelo por producto, el error se reduce a 18.27%.

4.3 Evaluación de la Seguridad y Estándares

4.4 Análisis de Impactos de la Solución

4.5 Conclusiones

4.6 Recomendaciones de Operación y Mantenibilidad

4.7 Trabajos Futuros

4.8 Declaración de Uso Responsable de IA

Referencias Bibliográficas

- [1] P. MEANA COALLA, *Gestión de inventarios* (Comercio y Marketing). Ediciones Paraninfo, S.A, 2024, ISBN: 9788428363570. [Internet]. available from <https://books.google.com.co/books?id=RfgyEQAAQBAJ>.
- [2] IBM, (n.d.). “¿Qué es la gestión de inventario?”, IBM Think. [Accessed: Aug. 5, 2026]. [Internet]. available from <https://www.ibm.com/mx-es/think/topics/inventory-management>.
- [3] Teaganne Finn , Ian Smalley, (n.d.). “¿Qué es la previsión de demanda?”, IBM Think. [Accessed: Aug. 5, 2026]. [Internet]. available from <https://www.ibm.com/es-es/think/topics/demand-forecasting>.
- [4] M. Valencia Cárdenas, F. J. Díaz Serna, and J. C. Correa Morales, “Inventory planning with dynamic demand. A state of art review”, DYNA, vol. 82, no. 190, pp. 182–191, Mar. 2015. doi: 10.15446/dyna.v82n190.42828 [Internet]. available from <https://revistas.unal.edu.co/index.php/dyna/article/view/42828>.
- [5] S. C. S. Fernando and O. B. C. Camilo, *Análisis de series temporales para predicción de demanda de productos*, 2024-11-05. [Internet]. available from <https://repository.unad.edu.co/handle/10596/69801>.
- [6] X. Wen and W. Li, “Time series prediction based on lstm-attention-lstm model”, IEEE Access, vol. 11, pp. 48 322–48 331, 2023. doi: 10.1109/ACCESS.2023.3276628
- [7] F. M. Salem, “Gated rnn: The gated recurrent unit (gru) rnn”, pp. 85–100, 2022. doi: 10.1007/978-3-030-89929-5_5 [Internet]. available from https://doi.org/10.1007/978-3-030-89929-5_5.
- [8] I. M. Peláez, “Modelos de regresión: Lineal simple y regresión logística”, Revista Seden, vol. 14, pp. 195–214, 2016.
- [9] X. Ren, H. Guo, S. Li, S. Wang, and J. Li, “A novel image classification method with cnn-xgboost model”, in *International workshop on digital watermarking*, Springer, 2017, pp. 378–390.

ANEXOS

Anexo A Documento completo SRS (historias de usuario / backlog)

TABLA VII
Especificación del caso de uso CU-01: Iniciar sesión

Elemento	Especificación
ID	CU-01
Nombre	Iniciar sesión
Actor principal	Usuario registrado
Requisitos relacionados	RF-10
Objetivo	El objetivo es permitir que los usuarios registrados accedan al sistema utilizando sus credenciales.
Precondiciones	Para poder iniciar sesión, el usuario debe encontrarse previamente registrado en el sistema.
Postcondiciones	El usuario accede exitosamente al sistema y puede acceder a las funcionalidades a las que tenga acceso dependiendo de su rol.
Flujo principal	1. El usuario ingresa su usuario y contraseña. 2. El sistema valida que la información sea correcta. 3. El sistema identifica el perfil del usuario. 4. El sistema permite el acceso a Thary.
Flujos alternativos	1. Si el usuario o la contraseña introducidos son incorrectas, el sistema le informa al usuario que los datos ingresados no son válidos. 2. Si el usuario no está registrado, el sistema rechaza el acceso y el usuario tiene la oportunidad de registrarse.

TABLA VIII
Especificación del caso de uso CU-02: Cargar historial de consumo

Elemento	Especificación
ID	CU-02
Nombre	Cargar historial de consumo
Actor principal	Administrador
Requisitos relacionados	RF-01, RF-07
Objetivo	El objetivo del caso de uso es permitirle al Administrador poder cargar un archivo CSV que contenga el historial de consumo de los productos del inventario.
Precondiciones	• Para acceder a esta funcionalidad, el usuario debe haber iniciado sesión con perfil de Administrador. • El archivo debe subirse en formato de CSV.
Postcondiciones	El archivo es recibido por el sistema para su posterior procesamiento
Flujo principal	1. El Administrador inicia sesión y es recibido por la pantalla inicial que le da la opción de subir un archivo csv. 2. El Administrador selecciona un archivo CSV. 3. El sistema recibe el archivo. 4. El sistema se encarga de validar el formato y que contenga los campos mínimos requeridos para el funcionamiento del sistema como: fecha, ID del producto y cantidad demandada de cada uno. 5. El sistema informa que el archivo fue cargado correctamente.
Flujos alternativos	1. Si el archivo no tiene formato CSV, el sistema informa el error con el mensaje "El archivo debe tener extensión .csv" 2. Si faltan campos obligatorios, el sistema rechaza el archivo e informa cuáles son los datos requeridos, ejemplo: Falta la columna obligatoria: fecha" 3. Si el usuario no cuenta con los permisos de Administrador, Thary simplemente no le muestra la opción de carga archivo, limitándose con solo mostrarle el dashboard con los resultados y demás pantallas con información.

TABLA IX
Especificación del caso de uso CU-03: Procesar archivo

Elemento	Especificación
ID	CU-03
Nombre	Procesar archivo
Actor principal	Administrador
Requisitos relacionados	RF-02, RF-07
Objetivo	Iniciar el procesamiento del historial de consumo previamente cargado para generar las estimaciones de demanda.
Precondiciones	• El Administrador debe haber iniciado sesión. • Debe existir un archivo de historial cargado y válido.
Postcondiciones	El sistema procesa los datos, ejecuta el proceso de predicción y genera los resultados de demanda para los productos.
Flujo principal	1. El Administrador accede al panel de procesamiento. 2. El sistema verifica que exista un archivo válido. 3. El Administrador selecciona la opción “Procesar archivo”. 4. El sistema procesa y prepara los datos. 5. El sistema ejecuta el proceso de predicción. 6. El sistema genera las estimaciones de demanda mensual para los productos. 7. El sistema genera los resultados necesarios para las etapas posteriores.
Flujos alternativos	1. Si no existe un archivo válido, el sistema informa al Administrador que debe cargar un historial. 2. Si ocurre un error durante el procesamiento, el sistema informa que no fue posible completar la operación. 3. Si el usuario no es Administrador, el sistema impide la ejecución.

TABLA X
Especificación del caso de uso CU-04: Consultar demanda pronosticada

Elemento	Especificación
ID	CU-04
Nombre	Consultar demanda pronosticada
Actor principal	Personal de Inventario
Requisitos relacionados	RF-02, RF-05
Objetivo	Permitirle al los empleados del inventario consultar cual es la demanda mensual estimada para cada uno de los producto.
Precondiciones	Como precondition, ya debe de existir un archivo procesado anteriormente por el administrador, tambien ya deben haber resultados de predicción disponibles.
Postcondiciones	El usuario visualiza las estimaciones de demanda correspondientes a los productos.
Flujo principal	1. El usuario accede al dashboard con las predicciones. 2. El sistema consulta los resultados generados. 3. El sistema le muestra al usuario, sin importar si es administrador o usuario, la demanda pronosticada de los productos. 4. El usuario consulta la información presentada.
Flujos alternativos	Si no existen resultados de predicción, el sistema informa que primero debe realizarse el procesamiento del archivo mediante el siguiente mensaje "Todavía no hay datos cargados. En cuanto tu administrador suba un archivo, vas a poder ver el pronóstico, el inventario y confirmar pedidos."

TABLA XI
Especificación del caso de uso CU-05: Consultar estado del inventario

Elemento	Especificación
ID	CU-05
Nombre	Consultar estado del inventario
Actor principal	Personal de Inventario
Requisitos relacionados	RF-04, RF-05
Objetivo	El objetivo es que se pueda consultar el inventario actual de los productos e identificar fácilmente a aquellos que requieran atención o estén en mayor riesgo de desabatecimiento.
Precondiciones	Como precondicion para este caso de uso, es importante que ya exista información del inventario disponible y resultados del procesamiento.
Postcondiciones	Las postcondiciones son que el usuario puede visualizar sin problemas el estado actual del inventario y las alertas correspondientes para cada producto.
Flujo principal	1. El usuario accede al módulo de "Estado de inventario". 2. El sistema consulta la informacion actual del inventario. 3. El sistema se encarga de comparar el inventario disponible con el punto de reposición de cada producto. 4. El sistema identifica los productos que se encuentran en o por debajo de su punto de reposición. 5. El sistema muestra las alertas visuales correspondientes dependiendo de cada caso, se marcaran de color rojo en caso de estar por debajo del punto de reposicion.
Flujos alternativos	Si no existe información de inventario para un producto, el sistema informa que no se dispone de los datos necesarios para determinar su estado.

TABLA XII

Especificación del caso de uso CU-06: Consultar recomendaciones de reposición y presupuesto

Elemento	Especificación
ID	CU-06
Nombre	Consultar recomendaciones de reposición y presupuesto
Actor principal	Personal de Inventario
Requisitos relacionados	RF-03, RF-05
Objetivo	Permitir consultar las recomendaciones de reposición generadas para cada producto, teniendo en cuenta distintos datos como la demanda estimada, el inventario que ya se tiene disponible, el tiempo de entrega, que varía de producto a producto, y el presupuesto establecido.
Precondiciones	La principal precondición es que debe existir un archivo csv ya procesado, y el sistema ya debe contar con toda la información necesaria para poder calcular las recomendaciones.
Postcondiciones	El usuario visualiza la cantidad recomendada de compra de una manera clara mediante el uso de gráficas y tablas, también puede ver otros datos relevantes como el punto de reposición y información acerca del presupuesto.
Flujo principal	1. El usuario accede al módulo de recomendaciones, es decir, el dashboard principal. 2. El sistema consulta las estimaciones de demanda. 3. El sistema obtiene el inventario actual y los parámetros de reposición. 4. El sistema considera el tiempo de entrega del proveedor y las restricciones presupuestales. 5. El sistema calcula la recomendación de reposición. 6. El sistema le muestra los resultados al usuario.
Flujos alternativos	Si no existen datos suficientes para realizar el cálculo, el sistema informa qué información se encuentra pendiente.

TABLA XIII
Especificación del caso de uso CU-07: Registrar pedido

Elemento	Especificación
ID	CU-07
Nombre	Registrar pedido
Actor principal	Personal de Inventario
Requisitos relacionados	RF-06
Objetivo	El objetivo es permitirle al Personal de Inventario registrar un pedido de reposición a partir de las recomendaciones generadas.
Precondiciones	El usuario debe haber iniciado sesión y debe existir información del producto y de la cantidad a solicitar.
Postcondiciones	El pedido queda registrado con la información correspondiente y asociado con el nombre del usuario responsable.
Flujo principal	1. El usuario consulta la pagina de "Pedidos" del sistema. 2. Selecciona uno de los pedidos para registrarlos. 3. Selecciona el proveedor. 4. A partir de esto, el sistema registra la fecha del pedido. 5. El sistema identifica al usuario responsable. 6. El sistema guarda la información del pedido. 7. El sistema confirma el registro.
Flujos alternativos	Si no se selecciona ningún producto, pero de todas maneras se hace click en el boton para registrar, saldra el siguiente mensaje: No seleccionaste ningún producto para confirmar"

TABLA XIV
Especificación del caso de uso CU-08: Gestionar usuarios

Elemento	Especificación
ID	CU-08
Nombre	Gestionar usuarios
Actor principal	Administrador
Requisitos relacionados	RF-08
Objetivo	Permitirle al Administrador poder crear cuentas para sus empleados con menos privilegios a los de un administrador.
Precondiciones	El usuario debe haber iniciado sesión con perfil de Administrador.
Postcondiciones	El nuevo empleado queda registrado y puede utilizar el sistema con permisos restringidos.
Flujo principal	1. El Administrador accede a la gestión de usuarios. 2. Selecciona la opción para crear un usuario. 3. Ingresa la información requerida. 4. Define el perfil o nivel de privilegios. 5. El sistema valida la información. 6. El sistema registra al nuevo usuario.
Flujos alternativos	1. Si la información es inválida o incompleta, el sistema solicita corregirla mediante un mensaje de error. 2. Si el usuario ya existe, el sistema informa que no puede crearse una cuenta duplicada.

TABLA XV
Especificación del caso de uso CU-09: Consultar asistente virtual

Elemento	Especificación
ID	CU-09
Nombre	Consultar asistente virtual
Actor principal	Usuario registrado
Requisitos relacionados	RF-10
Objetivo	El objetivo es permitirle a cualquier usuario del sistema resolver dudas sobre el funcionamiento de Thary mediante un asistente conversacional (chatbot), implementado utilizando la API de Ollama.
Precondiciones	• El usuario debe haber iniciado sesión en el sistema. • El servicio de Ollama debe estar disponible y en ejecución.
Postcondiciones	El usuario recibe una respuesta generada por el asistente correspondiente a su consulta.
Flujo principal	1. El usuario presiona el botón del asistente virtual, visible desde cualquier pantalla del sistema. 2. El sistema despliega una ventana emergente con el chat del asistente. 3. El usuario escribe su pregunta sobre el funcionamiento de Thary. 4. El sistema envía la consulta al servicio de Ollama mediante la API correspondiente. 5. El sistema recibe la respuesta generada por el modelo. 6. El sistema muestra la respuesta al usuario dentro de la ventana del chat.
Flujos alternativos	1. Si el servicio de Ollama no está corriendo o no responde dentro del tiempo esperado, el sistema informa al usuario que el asistente no se encuentra disponible en ese momento. 2. Si el usuario cierra la ventana del chat, el sistema descarta la conversación en curso sin afectar el resto de la sesión.

Anexo B Modelos de datos extensos, diccionarios de datos, mockups

Anexo C Manual de usuario y/o guía de instalación y despliegue

Anexo D Enlaces a repositorios de código fuente y tableros de gestión (Azure DevOps, Jira, GitHub)



Anexo E Protocolo y seguimiento de la revisión sistemática de literatura (si aplica)